

```

{
  "cells": [
    {
      "cell_type": "markdown",
      "metadata": {},
      "source": [
        "# Lecture 10 (Part b)\n"
      ]
    },
    {
      "cell_type": "markdown",
      "metadata": {},
      "source": [
        "\n",
        "## Reading Data\n",
        "\n",
        "We will start with playing around with a data file... lets make sure
it's there:"
      ]
    },
    {
      "cell_type": "code",
      "execution_count": 6,
      "metadata": {},
      "outputs": [
        {
          "name": "stdout",
          "output_type": "stream",
          "text": [
            "\u001b[31mLecture.10.a.ipynb\u001b[m\u001b[m M.pickle
file.json          file.yaml\r\n",
            "\u001b[31mLecture.10.b.ipynb\u001b[m\u001b[m M_list.pickle
file.pickle\r\n",
            "M.npy              \u001b[31mScores.csv\u001b[m\u001b[m
file.py\r\n"
          ]
        }
      ],
      "source": [
        "!ls "
      ]
    },
    {
      "cell_type": "code",
      "execution_count": 7,
      "metadata": {
        "scrolled": true
      },
      "outputs": [
        {
          "name": "stdout",
          "output_type": "stream",
          "text": [

```



```
"1,10,7,10,10,9.5,9.5,10,10,9.5,14,9.5,10,10,10,8,10,8,10,10,7,5,0,0,0,11
,10,10,10,10,5,6,0,0,0,0,0,1,10,15,9,9,10,9,8,9,7,9,10,10,10,10,0,0,0\r",
    "\r\n",
```

```
"1,10,7,10,10,0,0,10,10,7,14,10,10,10,10,7,10,6,3,10,10,10,10,10,10,11,10
,10,10,10,10,5,10,10,10,10,10,1,0,15,9,9,9,9,9,10,9,9,10,10,10,10,10,5,10
\r",
    "\r\n",
```

```
"1,10,7,10,10,9.5,0,10,10,0,14,9.5,0,0,10,0,10,5,10,7,0,10,6,10,0,11,10,1
0,6,0,0,0,0,0,0,0,0,1,0,15,0,0,0,0,5,0,7,0,3,3,3,0,3,0,0\r",
    "\r\n",
```

```
"1,10,7,0,0,0,0,0,0,0,14,0,0,0,0,0,0,0,0,0,0,0,0,0,0,11,0,0,0,0,0,0,0,0
,0,0,1,0,15,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0\r",
    "\r\n"
```

```
    ]
  }
],
"source": [
  "!cat Scores.csv"
]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "## Comma Separated Values (CSV) File Format\n",
    "\n",
    "The simplest and most common file format for storing data is called
    Comma Separated Values (CSV). Generally a CSV file represents a table,
    with the top row (first line of the file) consisting of the labels of the
    columns (separated by commas). Each column keeps a different feature or
    field. For example for student data, the first column could be the name,
    second the ID, third major, etc. After the first line, each row hold the
    data for one data point or example. In the case of student data, each row
    could correspond to one student."
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "### Reading CSV Files\n",
    "\n",
    "There are lots of libraries for reading CSV files into memory.
    Before we start using them, lets write our own. We'll need two
    things:\n",
    "\n",
    "* Means of reading and interpreting the file.\n",
    "* A representation of the read data in memory.\n",
    "\n",
    "Lets recall one of the ways in python to read a file."
  ]
}
```

[illegible]

```

        "\n",

"1,10,7,10,10,10,9.5,10,10,9.5,14,5,9.5,9.5,8,10,10,8,10,8,0,5,6,0,0,11,0
,10,10,10,0,5,0,0,0,0,0,1,9.5,15,9,9,10,9,9,10,7,0,9,9,9,0,5,0,0\n",
        "\n",

"1,10,7,10,10,9.5,0,10,10,0,14,9.5,10,10,10,10,10,10,10,0,0,10,5,10,10,11
,0,10,10,0,0,5,0,0,0,0,0,1,0,15,9,9,10,0,8,9,7,9,10,10,10,10,10,0,0\n",
        "\n",

"1,10,7,0,10,10,8,10,10,10,14,9,0,0,0,0,0,0,0,0,0,0,0,0,0,11,0,0,0,0,0,0,
0,0,0,0,0,1,9.5,15,9,9,0,9,8,0,0,0,0,0,0,0,0,0,0,0\n",
        "\n",

"1,10,7,10,10,9.5,9.5,10,10,9.5,14,9.5,10,10,10,8,10,8,10,10,7,5,0,0,0,11
,10,10,10,10,5,6,0,0,0,0,0,1,10,15,9,9,10,9,8,9,7,9,10,10,10,10,0,0,0\n",
        "\n",

"1,10,7,10,10,0,0,10,10,7,14,10,10,10,10,7,10,6,3,10,10,10,10,10,10,11,10
,10,10,10,10,5,10,10,10,10,10,1,0,15,9,9,9,9,9,10,9,9,10,10,10,10,10,5,10
\n",
        "\n",

"1,10,7,10,10,9.5,0,10,10,0,14,9.5,0,0,10,0,10,5,10,7,0,10,6,10,0,11,10,1
0,6,0,0,0,0,0,0,0,0,1,0,15,0,0,0,0,5,0,7,0,3,3,3,0,3,0,0\n",
        "\n",

"1,10,7,0,0,0,0,0,0,0,14,0,0,0,0,0,0,0,0,0,0,0,0,0,0,11,0,0,0,0,0,0,0,0
,0,0,1,0,15,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0\n",
        "\n"
    ]
}
],
"source": [
    "f=open(\"Scores.csv\", \"r\")\n",
    "\n",
    "first_line = f.readline()\n",
    "print(first_line)\n",
    "\n",
    "line = f.readline()\n",
    "while line:\n",
    "    print(line)\n",
    "    line = f.readline()\n",
    "\n",
    "f.close()"
]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "We successfully dumped the contents of the file, but we didn't \n",
        "\n",

```

```

    """ interpret it... each line is just a string, not a comma separated
list of keys or values. \n",
    """ or store it into memory... we just dumped it into the screen.\n",
    "\n",
    "Let's start on properly interpreting the first line, which is
special:"
]
},
{
    "cell_type": "code",
    "execution_count": 9,
    "metadata": {},
    "outputs": [
        {
            "name": "stdout",
            "output_type": "stream",
            "text": [
                '['11_n', '11_1', '12_n', '12_1', '12_2', '12_3', '12_4', '12_5',
'12_6', '12_7', '13_n', '13_1', '13_2', '13_3', '13_4', '13_5', '13_6',
'13_7', '13_8', '13_9', '13_10', '13_11', '13_12', '13_13', '13_14',
'14_n', '14_1', '14_2', '14_3', '14_4', '14_5', '14_6', '14_7', '14_8',
'14_9', '14_10', '14_11', 'q1_n', 'q1_1', 'e1_n', 'e1_1', 'e1_2', 'e1_3',
'e1_4', 'e1_5', 'e1_6', 'e1_7', 'e1_8', 'e1_9', 'e1_10', 'e1_11',
'e1_12', 'e1_13', 'e1_14', 'e1_15\\n']\n"
            ]
        }
    ],
    "source": [
        "f=open(\"Scores.csv\", \"r\")\n",
        "first_line = f.readline()\n",
        "print(first_line.split(\"\\n\", \"\\n\"))\n",
        "f.close()"
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "It appears that each line ends with `\\n`. Here's how we can remove
these newlines."
    ]
},
{
    "cell_type": "code",
    "execution_count": 10,
    "metadata": {},
    "outputs": [
        {
            "name": "stdout",
            "output_type": "stream",
            "text": [
                '['11_n', '11_1', '12_n', '12_1', '12_2', '12_3', '12_4', '12_5',
'12_6', '12_7', '13_n', '13_1', '13_2', '13_3', '13_4', '13_5', '13_6',
'13_7', '13_8', '13_9', '13_10', '13_11', '13_12', '13_13', '13_14',

```

```
'14_n', '14_1', '14_2', '14_3', '14_4', '14_5', '14_6', '14_7', '14_8',
'14_9', '14_10', '14_11', 'q1_n', 'q1_1', 'e1_n', 'e1_1', 'e1_2', 'e1_3',
'e1_4', 'e1_5', 'e1_6', 'e1_7', 'e1_8', 'e1_9', 'e1_10', 'e1_11',
'e1_12', 'e1_13', 'e1_14', 'e1_15']\n"
```

```
]
}
],
"source": [
    "f=open(\"Scores.csv\", \"r\")\n",
    "first_line = f.readline().rstrip()\n",
    "print(first_line.split(\"\", \"\"))\n",
    "f.close()"
]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "Finally lets store the first line, which is a list of the column
names:"
```

```
]
},
{
    "cell_type": "code",
    "execution_count": 11,
    "metadata": {},
    "outputs": [],
    "source": [
        "f=open(\"Scores.csv\", \"r\")\n",
        "first_line = f.readline().rstrip()\n",
        "keys=first_line.split(\"\", \"\")\n",
        "f.close()"
    ]
},
{
    "cell_type": "code",
    "execution_count": 12,
    "metadata": {
        "scrolled": true
    },
    "outputs": [
        {
            "data": {
                "text/plain": [
                    "'11_n',\n",
                    "'11_1',\n",
                    "'12_n',\n",
                    "'12_1',\n",
                    "'12_2',\n",
                    "'12_3',\n",
                    "'12_4',\n",
                    "'12_5',\n",
                    "'12_6',\n",
                    "'12_7',\n"
```

```

    " '13_n',\n",
    " '13_1',\n",
    " '13_2',\n",
    " '13_3',\n",
    " '13_4',\n",
    " '13_5',\n",
    " '13_6',\n",
    " '13_7',\n",
    " '13_8',\n",
    " '13_9',\n",
    " '13_10',\n",
    " '13_11',\n",
    " '13_12',\n",
    " '13_13',\n",
    " '13_14',\n",
    " '14_n',\n",
    " '14_1',\n",
    " '14_2',\n",
    " '14_3',\n",
    " '14_4',\n",
    " '14_5',\n",
    " '14_6',\n",
    " '14_7',\n",
    " '14_8',\n",
    " '14_9',\n",
    " '14_10',\n",
    " '14_11',\n",
    " 'q1_n',\n",
    " 'q1_1',\n",
    " 'e1_n',\n",
    " 'e1_1',\n",
    " 'e1_2',\n",
    " 'e1_3',\n",
    " 'e1_4',\n",
    " 'e1_5',\n",
    " 'e1_6',\n",
    " 'e1_7',\n",
    " 'e1_8',\n",
    " 'e1_9',\n",
    " 'e1_10',\n",
    " 'e1_11',\n",
    " 'e1_12',\n",
    " 'e1_13',\n",
    " 'e1_14',\n",
    " 'e1_15']"
  ]
},
"execution_count": 12,
"metadata": {},
"output_type": "execute_result"
}
],
"source": [
  "keys"

```



```
]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "Now lets read the rest of the file in a similar fashion:"
  ]
},
{
  "cell_type": "code",
  "execution_count": 13,
  "metadata": {},
  "outputs": [],
  "source": [
    "f=open(\"Scores.csv\", \"r\")\n",
    "first_line = f.readline().rstrip()\n",
    "keys=first_line.split(\",\")\n",
    "\n",
    "data=list()\n",
    "\n",
    "line = f.readline().rstrip()\n",
    "while line:\n",
    "    data.append(line.split(\",\"))\n",
    "    line = f.readline().rstrip()\n",
    "\n",
    "f.close()"
  ]
},
{
  "cell_type": "code",
  "execution_count": 14,
  "metadata": {},
  "outputs": [
    {
      "data": {
        "text/plain": [
          "['1',\n",
          " '10',\n",
          " '7',\n",
          " '10',\n",
          " '10',\n",
          " '9.5',\n",
          " '9',\n",
          " '10',\n",
          " '10',\n",
          " '9.5',\n",
          " '14',\n",
          " '10',\n",
          " '10',\n",
          " '10',\n",
          " '10',\n",
          " '10',\n",
          " '10']"
```

[illegible]

```
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '11',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '5',\n",
" '3',\n",
" '0',\n",
" '3',\n",
" '10',\n",
" '7',\n",
" '0',\n",
" '1',\n",
" '9.5',\n",
" '15',\n",
" '9',\n",
" '9',\n",
" '10',\n",
" '5',\n",
" '10',\n",
" '0',\n",
" '9',\n",
" '9',\n",
" '9',\n",
" '9',\n",
" '9',\n",
" '9',\n",
" '10',\n",
" '5',\n",
" '0',\n",
" '0'],\n",
" ['1',\n",
" '10',\n",
" '7',\n",
" '10',\n",
" '10',\n",
" '0',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '14',\n",
" '10',\n",
" '6',\n",
" '10',\n",
" '0',\n",
```

[illegible]

```
" '10',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '11',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '3',\n",
" '3',\n",
" '0',\n",
" '0',\n",
" '5',\n",
" '0',\n",
" '0',\n",
" '1',\n",
" '10',\n",
" '15',\n",
" '9',\n",
" '9',\n",
" '10',\n",
" '0',\n",
" '10',\n",
" '0',\n",
" '7',\n",
" '5',\n",
" '9',\n",
" '9',\n",
" '9',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0'],\n",
" ['1',\n",
" '10',\n",
" '7',\n",
" '10',\n",
" '10',\n",
" '5',\n",
" '9.5',\n",
" '10',\n",
" '10',\n",
" '9.5',\n",
" '14',\n",
" '5',\n",
" '9',
```

```
"  '9',\n",
"  '10',\n",
"  '7',\n",
"  '10',\n",
"  '10',\n",
"  '10',\n",
"  '10',\n",
"  '7',\n",
"  '10',\n",
"  '3',\n",
"  '5',\n",
"  '10',\n",
"  '11',\n",
"  '0',\n",
"  '0',\n",
"  '0',\n",
"  '0',\n",
"  '0',\n",
"  '0',\n",
"  '0',\n",
"  '0',\n",
"  '0',\n",
"  '0',\n",
"  '0',\n",
"  '0',\n",
"  '0',\n",
"  '0',\n",
"  '0',\n",
"  '0',\n",
"  '1',\n",
"  '10',\n",
"  '15',\n",
"  '9',\n",
"  '9',\n",
"  '9',\n",
"  '8',\n",
"  '7',\n",
"  '10',\n",
"  '0',\n",
"  '9',\n",
"  '10',\n",
"  '9',\n",
"  '10',\n",
"  '9',\n",
"  '5',\n",
"  '0',\n",
"  '0'],\n",
"  ['1',\n",
"  '10',\n",
"  '7',\n",
"  '10',\n",
"  '10',\n",
"  '3',\n",
"  '7',\n",
"  '10',\n",
"  '10',\n",
"  '9',\n",
"  '14',\n",
"  '10',
```

```
" '10',\n",
" '10',\n",
" '10',\n",
" '0',\n",
" '10',\n",
" '9',\n",
" '10',\n",
" '7',\n",
" '7',\n",
" '3',\n",
" '7',\n",
" '5',\n",
" '8',\n",
" '11',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '8',\n",
" '5',\n",
" '3',\n",
" '0',\n",
" '0',\n",
" '7',\n",
" '0',\n",
" '0',\n",
" '1',\n",
" '9.5',\n",
" '15',\n",
" '9',\n",
" '9',\n",
" '10',\n",
" '10',\n",
" '7',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '9',\n",
" '8',\n",
" '2'],\n" ['1',\n" '10',\n" '7',\n" '10',\n" '10',\n" '3',\n" '9.5',\n" '10',\n" '10',\n" '9.5',\n" '14',\n
```

```
" '10',\n",
" '10',\n",
" '10',\n",
" '8',\n",
" '5',\n",
" '10',\n",
" '5',\n",
" '10',\n",
" '3',\n",
" '0',\n",
" '10',\n",
" '3',\n",
" '10',\n",
" '8',\n",
" '11',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '0',\n",
" '0',\n",
" '10',\n",
" '5',\n",
" '0',\n",
" '1',\n",
" '10',\n",
" '15',\n",
" '9',\n",
" '9',\n",
" '10',\n",
" '9',\n",
" '7',\n",
" '9',\n",
" '0',\n",
" '0',\n",
" '10',\n",
" '10',\n",
" '9',\n",
" '5',\n",
" '10',\n",
" '8',\n",
" '0'],\n",
" ['1',\n",
" '10',\n",
" '7',\n",
" '10',\n",
" '10',\n",
" '0',\n",
" '5',\n",
" '10',\n",
" '10',\n",
" '9.5',\n",
```


[illegible]

[illegible]

```
" '10',\n",
" '9.5',\n",
" '14',\n",
" '5',\n",
" '9.5',\n",
" '9.5',\n",
" '8',\n",
" '10',\n",
" '10',\n",
" '8',\n",
" '10',\n",
" '8',\n",
" '0',\n",
" '5',\n",
" '6',\n",
" '0',\n",
" '0',\n",
" '11',\n",
" '0',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '0',\n",
" '5',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '1',\n",
" '9.5',\n",
" '15',\n",
" '9',\n",
" '9',\n",
" '10',\n",
" '9',\n",
" '9',\n",
" '10',\n",
" '7',\n",
" '0',\n",
" '9',\n",
" '9',\n",
" '9',\n",
" '0',\n",
" '5',\n",
" '0',\n",
" '0'],\n",
" ['1',\n",
" '10',\n",
" '7',\n",
" '10',\n",
" '10',\n",
" '9.5',\n",
" '0',
```

```
" '10',\n",
" '10',\n",
" '0',\n",
" '14',\n",
" '9.5',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '0',\n",
" '0',\n",
" '10',\n",
" '5',\n",
" '10',\n",
" '10',\n",
" '11',\n",
" '0',\n",
" '10',\n",
" '10',\n",
" '0',\n",
" '0',\n",
" '5',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '1',\n",
" '0',\n",
" '15',\n",
" '9',\n",
" '9',\n",
" '10',\n",
" '0',\n",
" '8',\n",
" '9',\n",
" '7',\n",
" '9',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '0',\n",
" '0'],\n",
" ['1',\n",
" '10',\n",
" '7',\n",
" '0',\n",
" '10',\n",
" '10',\n",
```

[illegible]

```
" '9.5',\n",  
" '9.5',\n",  
" '10',\n",  
" '10',\n",  
" '9.5',\n",  
" '14',\n",  
" '9.5',\n",  
" '10',\n",  
" '10',\n",  
" '10',\n",  
" '8',\n",  
" '10',\n",  
" '8',\n",  
" '10',\n",  
" '10',\n",  
" '7',\n",  
" '5',\n",  
" '0',\n",  
" '0',\n",  
" '0',\n",  
" '11',\n",  
" '10',\n",  
" '10',\n",  
" '10',\n",  
" '10',\n",  
" '5',\n",  
" '6',\n",  
" '0',\n",  
" '0',\n",  
" '0',\n",  
" '0',\n",  
" '0',\n",  
" '1',\n",  
" '10',\n",  
" '15',\n",  
" '9',\n",  
" '9',\n",  
" '10',\n",  
" '9',\n",  
" '8',\n",  
" '9',\n",  
" '7',\n",  
" '9',\n",  
" '10',\n",  
" '10',\n",  
" '10',\n",  
" '10',\n",  
" '0',\n",  
" '0',\n",  
" '0'],\n",  
" ['1',\n",  
" '10',\n",  
" '7',\n",  
" '10',
```

```
" '10',\n",
" '0',\n",
" '0',\n",
" '10',\n",
" '10',\n",
" '7',\n",
" '14',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '7',\n",
" '10',\n",
" '6',\n",
" '3',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '11',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '5',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '1',\n",
" '0',\n",
" '15',\n",
" '9',\n",
" '9',\n",
" '9',\n",
" '9',\n",
" '9',\n",
" '10',\n",
" '9',\n",
" '9',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '10',\n",
" '5',\n",
" '10'],\n",
" ['1',\n",
" '10',\n",
" '7',\n",
```

```
" '10',\n",
" '10',\n",
" '9.5',\n",
" '0',\n",
" '10',\n",
" '10',\n",
" '0',\n",
" '14',\n",
" '9.5',\n",
" '0',\n",
" '0',\n",
" '10',\n",
" '0',\n",
" '10',\n",
" '5',\n",
" '10',\n",
" '7',\n",
" '0',\n",
" '10',\n",
" '6',\n",
" '10',\n",
" '0',\n",
" '11',\n",
" '10',\n",
" '10',\n",
" '6',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '1',\n",
" '0',\n",
" '15',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '0',\n",
" '5',\n",
" '0',\n",
" '7',\n",
" '0',\n",
" '3',\n",
" '3',\n",
" '3',\n",
" '0',\n",
" '3',\n",
" '0',\n",
" '0',\n",
" '0'],\n",
" ['1',\n",
" '10',\n",
```



```

    },
    "execution_count": 14,
    "metadata": {},
    "output_type": "execute_result"
  }
],
"source": [
  "data"
]
},
{
  "cell_type": "code",
  "execution_count": 15,
  "metadata": {},
  "outputs": [
    {
      "data": {
        "text/plain": [
          "16"
        ]
      },
      "execution_count": 15,
      "metadata": {},
      "output_type": "execute_result"
    }
  ],
  "source": [
    "len(data)"
  ]
},
{
  "cell_type": "code",
  "execution_count": 16,
  "metadata": {},
  "outputs": [
    {
      "data": {
        "text/plain": [
          "55"
        ]
      },
      "execution_count": 16,
      "metadata": {},
      "output_type": "execute_result"
    }
  ],
  "source": [
    "len(data[3])"
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [

```

```

    "We have everything in memory now, how do we retrieve it?\n",
    "\n",
    "To associate specific keys to column numbers, we can do the
following"
]
},
{
  "cell_type": "code",
  "execution_count": 17,
  "metadata": {},
  "outputs": [
    {
      "data": {
        "text/plain": [
          "26"
        ]
      },
      "execution_count": 17,
      "metadata": {},
      "output_type": "execute_result"
    }
  ],
  "source": [
    "keys.index(\"l4_1\")"
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "So the 10th student \"l4_1\" grade is:"
  ]
},
{
  "cell_type": "code",
  "execution_count": 18,
  "metadata": {},
  "outputs": [
    {
      "data": {
        "text/plain": [
          "'0'"
        ]
      },
      "execution_count": 18,
      "metadata": {},
      "output_type": "execute_result"
    }
  ],
  "source": [
    "data[10][keys.index(\"l4_1\")]"
  ]
},
{

```

```

"cell_type": "markdown",
"metadata": {},
"source": [
    "Note that it's still a string... not a number:"
]
},
{
    "cell_type": "code",
    "execution_count": 19,
    "metadata": {},
    "outputs": [
        {
            "data": {
                "text/plain": [
                    "str"
                ]
            },
            "execution_count": 19,
            "metadata": {},
            "output_type": "execute_result"
        }
    ],
    "source": [
        "type(data[10][keys.index(\"l4_1\")])"
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "### Representing a table with Python Built-ins\n",
        "\n",
        "Lets quickly think of several ways of representing a table in
memory:\n",
        "1. A list of lists (i.e. a 2-D array or matrix) and a dictionary,
where each element of the \"matrix\" corresponds to a specific grade for
a specific student and the dictionary maps the name of the column to the
column index. For example `data[5][column_names[\"l1_5\"]` corresponds
to the 6th student's grade on lab 3 question 5.\n",
        "\n",
        "1. A list of dictionaries, where each element of the list is
corresponds to a row of the table and the dictionaries are keyed by the
column name. For example `data[5][\"l3_5\"]` corresponds to the 6th
student's grade on lab 3 question 5.\n",
        "\n",
        "\n",
        "Note that both methods above store the data, indexing the row first,
aka *row-wise*. If this was a large dataset and the expected access
pattern (how we plan to use the data), required looking at all
information in a row at a time, then such an implementation would be most
efficient. If we instead wanted to look the values a single column for
all rows, then we be better off with a *column-wise* representation:\n",
        "\n",

```

"3. A dictionary of lists, where each element of the dictionary corresponds to a column of data and the lists contain the data in that column. For example `data[\"l3\_5\"]` corresponds to the 6th student's grade on lab 3 question 5.\n",

"\n",

"For example, we can easily choose number 2 for our storage. We'll go through it slowly in a bit."

```
]
},
{
  "cell_type": "code",
  "execution_count": 20,
  "metadata": {},
  "outputs": [
    {
      "data": {
        "text/plain": [
          "'0'"
        ]
      },
      "execution_count": 20,
      "metadata": {},
      "output_type": "execute_result"
    }
  ],
  "source": [
    "f=open(\"Scores.csv\", \"r\")\n",
    "first_line = f.readline().rstrip()\n",
    "keys=first_line.split(\"\", \"\")\n",
    "\n",
    "data=list()\n",
    "\n",
    "line = f.readline().rstrip()\n",
    "while line:\n",
    "    data.append(dict(zip(keys, line.split(\"\", \"\"))))\n",
    "    line = f.readline().rstrip()\n",
    "\n",
    "f.close()\n",
    "data[5][\"l3_5\"]"
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "## Building a CSV Reader"
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "We have the basics down, but we have more things to consider:\n",

```

```

    """ We have written some example code, we should now write something
    that is general and we could use in different instances. \n",
    """ The fields can be different types: strings, numbers (integer or
    floating point). We should store the fields as the correct data type.\n",
    """ Still need to figure out a nice way to store the data in
    memory.\n",
    """ We might want to also be able to write out CSV files.\n",
    "\n",
    "We have some options on how to proceed:\n",
    "    1. **Function reader / python built-in data representation**- We
    could write a CSV reader function `my_data = read_csv(filename)` that
    given the filename of a CSV file, reads the data and returns it as a
    standard python data object. As we discussed above, there are various
    suitable representations for `my_data`, so we'll either have to pick one
    or provide some options to allow for other ones.\n",
    "    1. **One class for reading and storage**- Instead of a CSV reader
    function, we could create a CSV reader class. It will be instantiated
    with a CSV filename `my_file = csv_file(filename)`, so each instance
    would be uniquely connected to a specific file. It'll read the data into
    some representation that is kept private. We provide accessor methods to
    get to retrieve specific parts of the data, or the whole data as standard
    python data.\n",
    "    1. **Separate classes for reading and storing**- We can separate
    the concepts of a CSV reader and how we store the data. In this way, we
    could write other readers (e.g. Excel file reader) that would still use
    the same data storage. This is the option we are going to explore below."
]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "### File Handling Class\n",
        "\n",
        "Consider the following implementation of a file reader that leaves
        room for supporting other file formats:"
    ]
},
{
    "cell_type": "code",
    "execution_count": 21,
    "metadata": {},
    "outputs": [],
    "source": [
        "class DataFileHandler:\n",
        "    def __init__(self, extensions):\n",
        "        self.__extensions=extensions\n",
        "        \n",
        "    def check_extension(self, filename):\n",
        "        file_extension=filename.split(\".\")[-1]\n",
        "        return file_extension in self.__extensions\n",
        "\n",
        "    def _readfile(self, filename):\n",
        "        raise NotImplementedError
    ]
}

```

```

        \n",
        "    def readfile(self,filename,check_extension=True):\n",
        "        if not check_extension or
self.check_extension(filename):\n",
        "            return self._readfile(filename)        \n",
        "        else:\n",
        "            print(\"Error: filename {} does not match acceptable
extensions.\".format(filename))\n",
        "        \n",
        "    def _writefile(self,filename,data):\n",
        "        raise NotImplementedError\n",
        "        \n",
        "    def writefile(self,filename,data):\n",
        "        return self._writefile(filename,data)\n",
        "        \n",
        "        \n",
        "class CSVHandler(DataFileHandler):\n",
        "    def __init__(self):\n",
        "        #super(CSVHandler,self).__init__(["csv","CSV"])\n",
        "        DataFileHandler.__init__(self,["csv","CSV"])\n",
        "        \n",
        "    def _readfile(self,filename):\n",
        "        f=open(filename,\"r\")\n",
        "        first_line = f.readline().rstrip()\n",
        "        fields=first_line.split(",")\n",
        "\n",
        "        data=list()\n",
        "\n",
        "        line = f.readline().rstrip()\n",
        "        while line:\n",
        "            data.append(line.split(","))\n",
        "            line = f.readline().rstrip()\n",
        "\n",
        "        f.close()\n",
        "        \n",
        "        return fields,data\n",
        "        "
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "Note that this implementation doesn't do anything fancy with the
data representation. But does allow for multiple handlers to be created
to read various types of files.\n",
        "\n",
        "Lets test:"
    ]
},
{
    "cell_type": "code",
    "execution_count": 22,

```

```

"metadata": {},
"outputs": [],
"source": [
    "my_handler=CSVHandler()\n",
    "fields,data=my_handler.readfile(\"Scores.csv\") "
]
},
{
    "cell_type": "code",
    "execution_count": 23,
    "metadata": {},
    "outputs": [
        {
            "data": {
                "text/plain": [
                    "str"
                ]
            },
            "execution_count": 23,
            "metadata": {},
            "output_type": "execute_result"
        }
    ],
    "source": [
        "type(data[0][3])"
    ]
},
{
    "cell_type": "code",
    "execution_count": 24,
    "metadata": {},
    "outputs": [
        {
            "data": {
                "text/plain": [
                    "'10'"
                ]
            },
            "execution_count": 24,
            "metadata": {},
            "output_type": "execute_result"
        },
        {
            "name": "stderr",
            "output_type": "stream",
            "text": [
                "Bad pipe message: %s
[b'\xc8#\x17\xbe\xf6\x1c\x19\xa2\xa9\x06\xd1\xa2\x82\xdfN\xa4\xb3w\x00\x01\x00\x00\x00\x01\x00\x02\x00\x03\x00\x04\x00\x05\x00\x06\x00\x07\x00\x08\x00\t\x00\n\x00\b\x00\x0c\x00\r\x00\x0e\x00\x0f\x00\x10\x00\x11\x00\x12\x00\x13\x00\x14\x00\x15\x00\x16\x00\x17\x00\x18\x00\x19\x00\x1a\x00\x1b\x00/\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x00:\x00;\x00<\x00=\x00']\n",

```


"Bad pipe message: %s

[b'?\x00@\x00A\x00B\x00C\x00D\x00E\x00F\x00g\x00h\x00i\x00j\x00k\x00l\x00m\x00\x84\x00\x85\x00\x86\x00\x87\x00\x88\x00\x89\x00\x96\x00\x97\x00\x98\x00\x99\x00\x9a\x00\x9b\x00\x9c\x00\x9d\x00\x9e\x00\x9f']\n",

"Bad pipe message: %s

[b'\xfe\xdc\xeb\n\b5)\x8e\xcbW\xd3\x87\xe9\xb2g:\xc2\xae\x03\x00\x01|\x00\x00\x00\x01\x00\x02\x00\x03\x00\x04\x00\x05\x00\x06\x00\x07\x00\x08\x00\t\x00\n\x00\x0b\x00\x0c\x00\r\x00\x0e\x00\x0f\x00\x10\x00\x11\x00\x12\x00\x13\x00\x14\x00\x15\x00\x16\x00\x17\x00\x18\x00\x19\x00\x1a\x00\x1b\x00/\x000\x01\x02\x003\x04\x05\x006\x07\x08\x009\x00',
b';\x00<\x00=\x00>\x00?\x00@\x00A\x00B\x00C\x00D\x00E\x00F\x00g\x00h\x00i\x00j\x00k\x00l\x00m\x00\x84\x00\x85\x00\x86\x00\x87\x00\x88\x00\x89\x00\x96\x00\x97\x00\x98\x00\x99']\n",

"Bad pipe message: %s

[b'\x8d\xef9\x12\xbe\xeb#e\xef3D\xc4\x9b=3F\x97H\xb3\x00\x01|\x00\x00\x00\x01\x00\x02\x00\x03\x00\x04\x00\x05\x00\x06\x00\x07\x00\x08\x00\t\x00\n\x00\x0b\x00\x0c\x00\r\x00\x0e\x00\x0f\x00\x10\x00\x11\x00\x12\x00\x13\x00\x14\x00\x15\x00\x16\x00\x17\x00\x18\x00\x19\x00\x1a\x00\x1b\x00/\x000\x01\x02\x03\x04\x05\x006\x07\x08\x009\x00:\x00;\x00<\x00=\x00>\x00?\x00@\x00A\x00B\x00C\x00D\x00E\x00F\x00g\x00h\x00i\x00j\x00k\x00l\x00m\x00\x84\x00\x85\x00\x86\x00\x87\x00\x88\x00\x89\x00\x96\x00\x97\x00\x98\x00\x99\x00\x9a\x00\x9b\x00\x9c\x00\x9d\x00\x9e\x00\x9f\x00\xa0\x00\xa1\x00\xa2\x00\xa3\x00\xa4\x00\xa5\x00\xa6\x00\xa7\x00\xba\x00\xbb\x00\xbc\x00\xbd\x00\xbe\x00\xbf\x00\xc0\x00\xc1\x00\xc2\x00\xc3\x00\xc4\x00\xc5\x13\x01\x13\x02\x13\x03\x13\x04\x13\x05\x00\x01\x02\x00\x03\x00\x04\x00\x05\x00\x06\x00\x07\x00\x08\x00\t\x00\n\x00\x0b\x00\x0c\x00\r\x00',
b'\x0f\x00\x10\x00\x11\x00\x12\x00\x13\x00\x14\x00\x15']\n",

"Bad pipe message: %s

[b'\x80\x99\xd3"\x18\xffo\xb8\x9e\x98\x04\xf0\xb6Q\xe2[\xb0,\x00\x01|\x00\x00\x00\x01\x00\x02\x00\x03\x00\x04\x00\x05\x00\x06\x00\x07\x00\x08\x00\t\x00\n\x00\x0b\x00\x0c\x00\r\x00\x0e\x00\x0f\x00\x10\x00\x11\x00\x12\x00\x13\x00\x14\x00\x15\x00\x16\x00\x17\x00\x18\x00\x19\x00\x1a\x00\x1b\x00/\x000\x01\x02\x003']\n",

"Bad pipe message: %s

[b'\$\xfb\x1f\xc5\x8b\xf8\x8a\x9fDO\xf8\xf4IV\xf8b\xc7\x0c\x00\x01|\x00\x00\x00\x01\x00\x02\x00\x03\x00\x04\x00\x05\x00\x06\x00\x07\x00\x08\x00\t\x00\n\x00\x0b\x00\x0c\x00\r\x00\x0e\x00\x0f\x00\x10\x00\x11\x00\x12\x00\x13\x00\x14\x00\x15\x00\x16\x00\x17\x00\x18\x00\x19\x00\x1a\x00\x1b\x00/\x000\x01\x02\x03\x04\x05\x006\x07\x08\x009\x00:\x00;\x00<\x00=\x00>\x00?\x00@\x00A\x00B\x00C\x00D\x00E\x00F']\n",

"Bad pipe message: %s

[b'A7\xb0\xfed|\xa7\x9f\xf0:1V\x13L\x9d}\xa1\x81\x00\x01|\x00\x00\x00\x01\x00\x02\x00\x03\x00\x04\x00\x05\x00\x06\x00\x07\x00\x08\x00\t\x00\n\x00\x0b\x00\x0c\x00\r\x00\x0e\x00\x0f\x00\x10\x00\x11\x00\x12\x00\x13\x00\x14\x00\x15\x00\x16\x00\x17\x00\x18\x00\x19\x00\x1a\x00\x1b\x00/\x000\x01\x002\x03\x04\x05\x006\x07\x08\x009\x00:\x00;\x00<\x00=\x00>

```
\\x00?\\x00@\\x00A\\x00B\\x00C\\x00D\\x00E\\x00F\\x00g\\x00h\\x00i\\x00j\\x00k\\x00l\\x00m\\x00\\x84\\x00\\x85\\x00\\x86\\x00']\\n",
```

```
"Bad pipe message: %s
```

```
[b"\\x88\\x00\\x89\\x00\\x96\\x00\\x97\\x00\\x98\\x00\\x99\\x00\\x9a\\x00\\x9b\\x00\\x9c\\x00\\x9d\\x00\\x9e\\x00\\x9f\\x00\\xa0\\x00\\xa1\\x00\\xa2\\x00\\xa3\\x00\\xa4\\x00\\xa5\\x00\\xa6\\x00\\xa7\\x00\\xba\\x00\\xbb\\x00\\xbc\\x00\\xbd\\x00\\xbe\\x00\\xbf\\x00\\xc0\\x00\\xc1\\x00\\xc2\\x00\\xc3\\x00\\xc4\\x00\\xc5\\x13\\x01\\x13\\x02\\x13\\x03\\x13\\x04\\x13\\x05\\xc0\\x01\\xc0\\x02\\xc0\\x03\\xc0\\x04\\xc0\\x05\\xc0\\x06\\xc0\\x07\\xc0\\x08\\xc0\\t\\xc0\\n\\xc0\\x0b\\xc0\\x0c\\xc0\\r\\xc0\\x0e\\xc0\\x0f\\xc0\\x10\\xc0\\x11\\xc0\\x12\\xc0\\x13\\xc0\\x14\\xc0\\x15\\xc0\\x16\\xc0\\x17\\xc0\\x18\\xc0\\x19\\xc0#\\xc0$\\xc0%\\xc0&\\xc0'\\xc0"]\\n",
```

```
"Bad pipe message: %s [b')\\xc0*\\xc0+\\xc0,\\xc0-
```

```
\\xc0.\\xc0/\\xc00\\xc01\\xc02\\xc0r\\xc0s\\xc0t\\xc0u\\xc0v\\xc0w\\xc0x\\xc0y\\xc0z\\xc0{'']\\n"
```

```
]
```

```
}
```

```
],
```

```
"source": [
```

```
"data[0][3]"
```

```
]
```

```
},
```

```
{
```

```
"cell_type": "markdown",
```

```
"metadata": {},
```

```
"source": [
```

```
"#### Handling Different Types\\n",
```

```
"\\n",
```

```
"When we read a text file, all the content is assumed to be composed of strings. Instead, ideally we would like to interpret the file by looking at each field and selecting an appropriate type."
```

```
]
```

```
},
```

```
{
```

```
"cell_type": "markdown",
```

```
"metadata": {},
```

```
"source": [
```

```
"So we need a way to check if a string can be converted to a number (e.g. float). \\n",
```

```
"\\n",
```

```
"A way to check if a string can be converted to a number is to simply attempt to make it into a float, if it succeeds, then all is well, if not, we'll get an error which we'll catch and handle appropriately.\\n",
```

```
"\\n",
```

```
"What is the error we get? Lets induce the error and see what python says:"
```

```
]
```

```
},
```

```
{
```

```
"cell_type": "code",
```

```
"execution_count": 2,
```

```
"metadata": {
```

```
"scrolled": true
```

```
},
```

```

    "outputs": [
      {
        "ename": "ValueError",
        "evalue": "could not convert string to float: 'foo'",
        "output_type": "error",
        "traceback": [
          "\u001b[0;31m-----\n-----\u001b[0m",
          "\u001b[0;31mValueError\u001b[0m\nTraceback (most recent call last)",
          "Cell \u001b[0;32mIn[2], line 1\u001b[0m\n\u001b[0;32m---->\n1\u001b[0m\n\u001b[0;38;5;28;43mfloat\u001b[39;49m\u001b[43m(\u001b[49m\u001b[38;5;124;43m\u001b[39;49m\u001b[38;5;124;43mfoo\u001b[39;49m\u001b[38;5;124;43m\u001b[39;49m\u001b[43m)\u001b[49m\n",
          "\u001b[0;31mValueError\u001b[0m: could not convert string to\nfloat: 'foo'"
        ]
      },
      {
        "source": [
          "float(\"foo\")"
        ]
      },
      {
        "cell_type": "markdown",
        "metadata": {},
        "source": [
          "`ValueError` is the type of error we'll catch. And when the\nconversion works, there is no error:"
        ]
      },
      {
        "cell_type": "code",
        "execution_count": 3,
        "metadata": {},
        "outputs": [
          {
            "data": {
              "text/plain": [
                "5.0"
              ]
            },
            "execution_count": 3,
            "metadata": {},
            "output_type": "execute_result"
          }
        ],
        "source": [
          "float(5)"
        ]
      },
      {
        "cell_type": "markdown",

```

```

"metadata": {},
"source": [
    "Here's an example of how we can catch the error:"
]
},
{
    "cell_type": "code",
    "execution_count": 4,
    "metadata": {},
    "outputs": [
        {
            "name": "stdout",
            "output_type": "stream",
            "text": [
                "x: 5.0 <class 'float'>\n",
                "y: foo <class 'str'>\n"
            ]
        }
    ],
    "source": [
        "x=\"5\"\n",
        "y=\"foo\"\n",
        "\n",
        "try:\n",
        "    x=float(x)\n",
        "except ValueError:\n",
        "    pass\n",
        "\n",
        "try:\n",
        "    y=float(y)\n",
        "except ValueError:\n",
        "    pass\n",
        "\n",
        "print(\"x:\",x,type(x))\n",
        "print(\"y:\",y,type(y))"
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "Lets add this to our `CSVHandler`:"
    ]
},
{
    "cell_type": "code",
    "execution_count": 1,
    "metadata": {},
    "outputs": [],
    "source": [
        "class DataFileHandler:\n",
        "    def __init__(self,extensions):\n",
        "        self.__extensions=extensions\n",
        "        \n",

```

```

    def check_extension(self, filename):\n",
    def _readfile(self, filename):\n",
    def readfile(self, filename, check_extension=True):\n",
self.check_extension(filename):\n",
    return self._readfile(filename)
    else:\n",
    print("Error: filename {} does not match acceptable
extensions.\n".format(filename))\n",
    def _writefile(self, filename, data):\n",
    def writefile(self, filename, data):\n",
    return self._writefile(filename, data)\n",
class CSVHandler(DataFileHandler):\n",
    def __init__(self):\n",
    #super(CSVHandler, self).__init__(["csv", "CSV"])\n",
    DataFileHandler.__init__(self, ["csv", "CSV"])\n",
    def _readfile(self, filename):\n",
    f=open(filename, "r")\n",
    first_line = f.readline().rstrip()\n",
    fields=first_line.split(",")\n",
    data=list()\n",
    line = f.readline().rstrip()\n",
    while line:\n",
    items=line.split(",")\n",
    row=list()\n",
    for item in items:\n",
    # Using Error Catching (Exception handling) to test
type\n",
    try:\n",
    d=float(item)\n",
    except ValueError:\n",
    d=item\n",
    row.append(d)\n",
    data.append(row)\n",
    line = f.readline().rstrip()\n",
    f.close()\n",

```

```

        "        return fields,data\n",
        "        \n",
        "    "
    ]
},
{
    "cell_type": "code",
    "execution_count": 4,
    "metadata": {},
    "outputs": [],
    "source": [
        "my_handler=CSVHandler()\n",
        "fields,data=my_handler.readfile(\"Scores.csv\") "
    ]
},
{
    "cell_type": "code",
    "execution_count": 5,
    "metadata": {},
    "outputs": [
        {
            "data": {
                "text/plain": [
                    "float"
                ]
            },
            "execution_count": 5,
            "metadata": {},
            "output_type": "execute_result"
        }
    ],
    "source": [
        "type(data[0][3])"
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "### Data Representation with python built-ins"
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "Since we are using a list of lists to contain the data of the CSV file in memory, we have to do a bit of manipulation to find specific fields in the list of lists:"
    ]
},
{
    "cell_type": "code",
    "execution_count": 6,

```

```

"metadata": {
  "scrolled": true
},
"outputs": [
  {
    "data": {
      "text/plain": [
        "0.0"
      ]
    },
    "execution_count": 6,
    "metadata": {},
    "output_type": "execute_result"
  }
],
"source": [
  "data[10][fields.index(\"l4_1\")]"
]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "As discussed above, we should use a dictionary instead... recall
some basics:"
  ]
},
{
  "cell_type": "code",
  "execution_count": 7,
  "metadata": {
    "scrolled": true
  },
  "outputs": [
    {
      "data": {
        "text/plain": [
          "{ 'L1': 1, 'L2': 2}"
        ]
      },
      "execution_count": 7,
      "metadata": {},
      "output_type": "execute_result"
    }
  ],
  "source": [
    "foo=dict()\n",
    "foo[\"L1\"] = 1\n",
    "foo[\"L2\"] = 2\n",
    "\n",
    "foo"
  ]
},
{

```

```

"cell_type": "code",
"execution_count": 8,
"metadata": {
  "scrolled": true
},
"outputs": [
  {
    "data": {
      "text/plain": [
        "1"
      ]
    },
    "execution_count": 8,
    "metadata": {},
    "output_type": "execute_result"
  }
],
"source": [
  "foo[\"L1\"]"
]
},
{
  "cell_type": "code",
  "execution_count": 9,
  "metadata": {
    "scrolled": true
  },
  "outputs": [
    {
      "data": {
        "text/plain": [
          "'L1': 1, 'L2': 2"
        ]
      },
      "execution_count": 9,
      "metadata": {},
      "output_type": "execute_result"
    }
  ],
  "source": [
    "dict([ (\"L1\",1), (\"L2\",2) ])"
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "So in principle, we can then take every row and convert it easily to
a dictionary:"
  ]
},
{
  "cell_type": "code",
  "execution_count": 10,

```



```

"metadata": {},
"outputs": [],
"source": [
    "first_row=dict(list(zip(fields,data[0])))"
]
},
{
    "cell_type": "code",
    "execution_count": 11,
    "metadata": {
        "scrolled": true
    },
    "outputs": [
        {
            "data": {
                "text/plain": [
                    "{ 'l1_n': 1.0, \n",
                    " 'l1_1': 10.0, \n",
                    " 'l2_n': 7.0, \n",
                    " 'l2_1': 10.0, \n",
                    " 'l2_2': 10.0, \n",
                    " 'l2_3': 9.5, \n",
                    " 'l2_4': 9.0, \n",
                    " 'l2_5': 10.0, \n",
                    " 'l2_6': 10.0, \n",
                    " 'l2_7': 9.5, \n",
                    " 'l3_n': 14.0, \n",
                    " 'l3_1': 10.0, \n",
                    " 'l3_2': 10.0, \n",
                    " 'l3_3': 10.0, \n",
                    " 'l3_4': 10.0, \n",
                    " 'l3_5': 10.0, \n",
                    " 'l3_6': 10.0, \n",
                    " 'l3_7': 9.0, \n",
                    " 'l3_8': 10.0, \n",
                    " 'l3_9': 3.0, \n",
                    " 'l3_10': 0.0, \n",
                    " 'l3_11': 3.0, \n",
                    " 'l3_12': 3.0, \n",
                    " 'l3_13': 5.0, \n",
                    " 'l3_14': 2.0, \n",
                    " 'l4_n': 11.0, \n",
                    " 'l4_1': 0.0, \n",
                    " 'l4_2': 0.0, \n",
                    " 'l4_3': 0.0, \n",
                    " 'l4_4': 0.0, \n",
                    " 'l4_5': 0.0, \n",
                    " 'l4_6': 0.0, \n",
                    " 'l4_7': 0.0, \n",
                    " 'l4_8': 0.0, \n",
                    " 'l4_9': 0.0, \n",
                    " 'l4_10': 0.0, \n",
                    " 'l4_11': 0.0, \n",
                    " 'q1_n': 1.0, \n",

```

```

        " 'q1_1': 0.0,\n",
        " 'e1_n': 15.0,\n",
        " 'e1_1': 9.0,\n",
        " 'e1_2': 9.0,\n",
        " 'e1_3': 10.0,\n",
        " 'e1_4': 5.0,\n",
        " 'e1_5': 5.0,\n",
        " 'e1_6': 0.0,\n",
        " 'e1_7': 0.0,\n",
        " 'e1_8': 10.0,\n",
        " 'e1_9': 10.0,\n",
        " 'e1_10': 10.0,\n",
        " 'e1_11': 10.0,\n",
        " 'e1_12': 0.0,\n",
        " 'e1_13': 10.0,\n",
        " 'e1_14': 5.0,\n",
        " 'e1_15': 10.0}"
    ]
},
"execution_count": 11,
"metadata": {},
"output_type": "execute_result"
}
],
"source": [
    "first_row"
]
},
{
    "cell_type": "code",
    "execution_count": 12,
    "metadata": {},
    "outputs": [
        {
            "data": {
                "text/plain": [
                    "0.0"
                ]
            },
            "execution_count": 12,
            "metadata": {},
            "output_type": "execute_result"
        }
    ],
    "source": [
        "first_row[\"l4_1\"]"
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "And then store the dictionary for each row in a list:"
    ]
}

```

```

},
{
  "cell_type": "code",
  "execution_count": 13,
  "metadata": {},
  "outputs": [],
  "source": [
    "new_data=list()\n",
    "\n",
    "for row in data:\n",
    "    new_data.append(dict(list(zip(fields,row))))"
  ]
},
{
  "cell_type": "code",
  "execution_count": 14,
  "metadata": {
    "scrolled": true
  },
  "outputs": [
    {
      "data": {
        "text/plain": [
          "[{'11_n': 1.0,\n",
          " '11_1': 10.0,\n",
          " '12_n': 7.0,\n",
          " '12_1': 10.0,\n",
          " '12_2': 10.0,\n",
          " '12_3': 9.5,\n",
          " '12_4': 9.0,\n",
          " '12_5': 10.0,\n",
          " '12_6': 10.0,\n",
          " '12_7': 9.5,\n",
          " '13_n': 14.0,\n",
          " '13_1': 10.0,\n",
          " '13_2': 10.0,\n",
          " '13_3': 10.0,\n",
          " '13_4': 10.0,\n",
          " '13_5': 10.0,\n",
          " '13_6': 10.0,\n",
          " '13_7': 9.0,\n",
          " '13_8': 10.0,\n",
          " '13_9': 3.0,\n",
          " '13_10': 0.0,\n",
          " '13_11': 3.0,\n",
          " '13_12': 3.0,\n",
          " '13_13': 5.0,\n",
          " '13_14': 2.0,\n",
          " '14_n': 11.0,\n",
          " '14_1': 0.0,\n",
          " '14_2': 0.0,\n",
          " '14_3': 0.0,\n",
          " '14_4': 0.0,\n",
          " '14_5': 0.0,\n",

```

```

"  'l4_6': 0.0,\n",
"  'l4_7': 0.0,\n",
"  'l4_8': 0.0,\n",
"  'l4_9': 0.0,\n",
"  'l4_10': 0.0,\n",
"  'l4_11': 0.0,\n",
"  'q1_n': 1.0,\n",
"  'q1_1': 0.0,\n",
"  'e1_n': 15.0,\n",
"  'e1_1': 9.0,\n",
"  'e1_2': 9.0,\n",
"  'e1_3': 10.0,\n",
"  'e1_4': 5.0,\n",
"  'e1_5': 5.0,\n",
"  'e1_6': 0.0,\n",
"  'e1_7': 0.0,\n",
"  'e1_8': 10.0,\n",
"  'e1_9': 10.0,\n",
"  'e1_10': 10.0,\n",
"  'e1_11': 10.0,\n",
"  'e1_12': 0.0,\n",
"  'e1_13': 10.0,\n",
"  'e1_14': 5.0,\n",
"  'e1_15': 10.0},\n",
" {'l11_n': 1.0,\n",
"  'l11_1': 10.0,\n",
"  'l12_n': 7.0,\n",
"  'l12_1': 0.0,\n",
"  'l12_2': 10.0,\n",
"  'l12_3': 9.5,\n",
"  'l12_4': 0.0,\n",
"  'l12_5': 10.0,\n",
"  'l12_6': 10.0,\n",
"  'l12_7': 0.0,\n",
"  'l13_n': 14.0,\n",
"  'l13_1': 10.0,\n",
"  'l13_2': 10.0,\n",
"  'l13_3': 10.0,\n",
"  'l13_4': 10.0,\n",
"  'l13_5': 0.0,\n",
"  'l13_6': 0.0,\n",
"  'l13_7': 0.0,\n",
"  'l13_8': 0.0,\n",
"  'l13_9': 0.0,\n",
"  'l13_10': 0.0,\n",
"  'l13_11': 0.0,\n",
"  'l13_12': 0.0,\n",
"  'l13_13': 0.0,\n",
"  'l13_14': 0.0,\n",
"  'l14_n': 11.0,\n",
"  'l14_1': 10.0,\n",
"  'l14_2': 10.0,\n",
"  'l14_3': 10.0,\n",
"  'l14_4': 10.0,\n",

```

```

"  'l4_5': 5.0, \n",
"  'l4_6': 3.0, \n",
"  'l4_7': 0.0, \n",
"  'l4_8': 3.0, \n",
"  'l4_9': 10.0, \n",
"  'l4_10': 7.0, \n",
"  'l4_11': 0.0, \n",
"  'q1_n': 1.0, \n",
"  'q1_1': 9.5, \n",
"  'e1_n': 15.0, \n",
"  'e1_1': 9.0, \n",
"  'e1_2': 9.0, \n",
"  'e1_3': 10.0, \n",
"  'e1_4': 5.0, \n",
"  'e1_5': 10.0, \n",
"  'e1_6': 0.0, \n",
"  'e1_7': 9.0, \n",
"  'e1_8': 9.0, \n",
"  'e1_9': 9.0, \n",
"  'e1_10': 9.0, \n",
"  'e1_11': 9.0, \n",
"  'e1_12': 10.0, \n",
"  'e1_13': 5.0, \n",
"  'e1_14': 0.0, \n",
"  'e1_15': 0.0}, \n",
" {'l1_n': 1.0, \n",
"  'l1_1': 10.0, \n",
"  'l2_n': 7.0, \n",
"  'l2_1': 10.0, \n",
"  'l2_2': 10.0, \n",
"  'l2_3': 0.0, \n",
"  'l2_4': 10.0, \n",
"  'l2_5': 10.0, \n",
"  'l2_6': 10.0, \n",
"  'l2_7': 10.0, \n",
"  'l3_n': 14.0, \n",
"  'l3_1': 10.0, \n",
"  'l3_2': 6.0, \n",
"  'l3_3': 10.0, \n",
"  'l3_4': 0.0, \n",
"  'l3_5': 0.0, \n",
"  'l3_6': 0.0, \n",
"  'l3_7': 0.0, \n",
"  'l3_8': 0.0, \n",
"  'l3_9': 0.0, \n",
"  'l3_10': 0.0, \n",
"  'l3_11': 0.0, \n",
"  'l3_12': 0.0, \n",
"  'l3_13': 0.0, \n",
"  'l3_14': 0.0, \n",
"  'l4_n': 11.0, \n",
"  'l4_1': 10.0, \n",
"  'l4_2': 10.0, \n",
"  'l4_3': 0.0, \n",

```

```
"  'l4_4': 7.0, \n",
"  'l4_5': 0.0, \n",
"  'l4_6': 0.0, \n",
"  'l4_7': 0.0, \n",
"  'l4_8': 0.0, \n",
"  'l4_9': 0.0, \n",
"  'l4_10': 0.0, \n",
"  'l4_11': 0.0, \n",
"  'q1_n': 1.0, \n",
"  'q1_1': 9.5, \n",
"  'e1_n': 15.0, \n",
"  'e1_1': 9.0, \n",
"  'e1_2': 9.0, \n",
"  'e1_3': 10.0, \n",
"  'e1_4': 9.0, \n",
"  'e1_5': 5.0, \n",
"  'e1_6': 9.0, \n",
"  'e1_7': 7.0, \n",
"  'e1_8': 9.0, \n",
"  'e1_9': 10.0, \n",
"  'e1_10': 10.0, \n",
"  'e1_11': 10.0, \n",
"  'e1_12': 5.0, \n",
"  'e1_13': 10.0, \n",
"  'e1_14': 5.0, \n",
"  'e1_15': 0.0}, \n",
" {'l1_n': 1.0, \n",
"  'l1_1': 10.0, \n",
"  'l2_n': 7.0, \n",
"  'l2_1': 10.0, \n",
"  'l2_2': 10.0, \n",
"  'l2_3': 10.0, \n",
"  'l2_4': 9.5, \n",
"  'l2_5': 10.0, \n",
"  'l2_6': 10.0, \n",
"  'l2_7': 9.5, \n",
"  'l3_n': 14.0, \n",
"  'l3_1': 10.0, \n",
"  'l3_2': 10.0, \n",
"  'l3_3': 10.0, \n",
"  'l3_4': 10.0, \n",
"  'l3_5': 0.0, \n",
"  'l3_6': 0.0, \n",
"  'l3_7': 0.0, \n",
"  'l3_8': 0.0, \n",
"  'l3_9': 0.0, \n",
"  'l3_10': 0.0, \n",
"  'l3_11': 0.0, \n",
"  'l3_12': 0.0, \n",
"  'l3_13': 0.0, \n",
"  'l3_14': 0.0, \n",
"  'l4_n': 11.0, \n",
"  'l4_1': 10.0, \n",
"  'l4_2': 10.0, \n",
```

```

"  'l4_3': 10.0,\n",
"  'l4_4': 10.0,\n",
"  'l4_5': 3.0,\n",
"  'l4_6': 3.0,\n",
"  'l4_7': 0.0,\n",
"  'l4_8': 0.0,\n",
"  'l4_9': 5.0,\n",
"  'l4_10': 0.0,\n",
"  'l4_11': 0.0,\n",
"  'q1_n': 1.0,\n",
"  'q1_1': 10.0,\n",
"  'e1_n': 15.0,\n",
"  'e1_1': 9.0,\n",
"  'e1_2': 9.0,\n",
"  'e1_3': 10.0,\n",
"  'e1_4': 0.0,\n",
"  'e1_5': 10.0,\n",
"  'e1_6': 0.0,\n",
"  'e1_7': 7.0,\n",
"  'e1_8': 5.0,\n",
"  'e1_9': 9.0,\n",
"  'e1_10': 9.0,\n",
"  'e1_11': 9.0,\n",
"  'e1_12': 0.0,\n",
"  'e1_13': 0.0,\n",
"  'e1_14': 0.0,\n",
"  'e1_15': 0.0},\n",
" {'l1_n': 1.0,\n",
"  'l1_1': 10.0,\n",
"  'l2_n': 7.0,\n",
"  'l2_1': 10.0,\n",
"  'l2_2': 10.0,\n",
"  'l2_3': 5.0,\n",
"  'l2_4': 9.5,\n",
"  'l2_5': 10.0,\n",
"  'l2_6': 10.0,\n",
"  'l2_7': 9.5,\n",
"  'l3_n': 14.0,\n",
"  'l3_1': 5.0,\n",
"  'l3_2': 9.0,\n",
"  'l3_3': 9.0,\n",
"  'l3_4': 10.0,\n",
"  'l3_5': 7.0,\n",
"  'l3_6': 10.0,\n",
"  'l3_7': 10.0,\n",
"  'l3_8': 10.0,\n",
"  'l3_9': 10.0,\n",
"  'l3_10': 7.0,\n",
"  'l3_11': 10.0,\n",
"  'l3_12': 3.0,\n",
"  'l3_13': 5.0,\n",
"  'l3_14': 10.0,\n",
"  'l4_n': 11.0,\n",
"  'l4_1': 0.0,\n",

```

```
" 'l4_2': 0.0, \n",
" 'l4_3': 0.0, \n",
" 'l4_4': 0.0, \n",
" 'l4_5': 0.0, \n",
" 'l4_6': 0.0, \n",
" 'l4_7': 0.0, \n",
" 'l4_8': 0.0, \n",
" 'l4_9': 0.0, \n",
" 'l4_10': 0.0, \n",
" 'l4_11': 0.0, \n",
" 'q1_n': 1.0, \n",
" 'q1_1': 10.0, \n",
" 'e1_n': 15.0, \n",
" 'e1_1': 9.0, \n",
" 'e1_2': 9.0, \n",
" 'e1_3': 9.0, \n",
" 'e1_4': 8.0, \n",
" 'e1_5': 7.0, \n",
" 'e1_6': 10.0, \n",
" 'e1_7': 0.0, \n",
" 'e1_8': 9.0, \n",
" 'e1_9': 10.0, \n",
" 'e1_10': 9.0, \n",
" 'e1_11': 10.0, \n",
" 'e1_12': 9.0, \n",
" 'e1_13': 5.0, \n",
" 'e1_14': 0.0, \n",
" 'e1_15': 0.0}, \n",
" {'l1_n': 1.0, \n",
" 'l1_1': 10.0, \n",
" 'l2_n': 7.0, \n",
" 'l2_1': 10.0, \n",
" 'l2_2': 10.0, \n",
" 'l2_3': 3.0, \n",
" 'l2_4': 7.0, \n",
" 'l2_5': 10.0, \n",
" 'l2_6': 10.0, \n",
" 'l2_7': 9.0, \n",
" 'l3_n': 14.0, \n",
" 'l3_1': 10.0, \n",
" 'l3_2': 10.0, \n",
" 'l3_3': 10.0, \n",
" 'l3_4': 10.0, \n",
" 'l3_5': 0.0, \n",
" 'l3_6': 10.0, \n",
" 'l3_7': 9.0, \n",
" 'l3_8': 10.0, \n",
" 'l3_9': 7.0, \n",
" 'l3_10': 7.0, \n",
" 'l3_11': 3.0, \n",
" 'l3_12': 7.0, \n",
" 'l3_13': 5.0, \n",
" 'l3_14': 8.0, \n",
" 'l4_n': 11.0, \n
```



```
"  'l4_1': 10.0, \n",
"  'l4_2': 10.0, \n",
"  'l4_3': 10.0, \n",
"  'l4_4': 8.0, \n",
"  'l4_5': 5.0, \n",
"  'l4_6': 3.0, \n",
"  'l4_7': 0.0, \n",
"  'l4_8': 0.0, \n",
"  'l4_9': 7.0, \n",
"  'l4_10': 0.0, \n",
"  'l4_11': 0.0, \n",
"  'q1_n': 1.0, \n",
"  'q1_1': 9.5, \n",
"  'e1_n': 15.0, \n",
"  'e1_1': 9.0, \n",
"  'e1_2': 9.0, \n",
"  'e1_3': 10.0, \n",
"  'e1_4': 10.0, \n",
"  'e1_5': 7.0, \n",
"  'e1_6': 10.0, \n",
"  'e1_7': 10.0, \n",
"  'e1_8': 10.0, \n",
"  'e1_9': 10.0, \n",
"  'e1_10': 10.0, \n",
"  'e1_11': 10.0, \n",
"  'e1_12': 10.0, \n",
"  'e1_13': 9.0, \n",
"  'e1_14': 8.0, \n",
"  'e1_15': 2.0}, \n",
" {'l1_n': 1.0, \n",
"  'l1_1': 10.0, \n",
"  'l2_n': 7.0, \n",
"  'l2_1': 10.0, \n",
"  'l2_2': 10.0, \n",
"  'l2_3': 3.0, \n",
"  'l2_4': 9.5, \n",
"  'l2_5': 10.0, \n",
"  'l2_6': 10.0, \n",
"  'l2_7': 9.5, \n",
"  'l3_n': 14.0, \n",
"  'l3_1': 10.0, \n",
"  'l3_2': 10.0, \n",
"  'l3_3': 10.0, \n",
"  'l3_4': 8.0, \n",
"  'l3_5': 5.0, \n",
"  'l3_6': 10.0, \n",
"  'l3_7': 5.0, \n",
"  'l3_8': 10.0, \n",
"  'l3_9': 3.0, \n",
"  'l3_10': 0.0, \n",
"  'l3_11': 10.0, \n",
"  'l3_12': 3.0, \n",
"  'l3_13': 10.0, \n",
"  'l3_14': 8.0, \n
```

```
"  'l4_n': 11.0, \n",
"  'l4_1': 10.0, \n",
"  'l4_2': 10.0, \n",
"  'l4_3': 10.0, \n",
"  'l4_4': 10.0, \n",
"  'l4_5': 10.0, \n",
"  'l4_6': 10.0, \n",
"  'l4_7': 0.0, \n",
"  'l4_8': 0.0, \n",
"  'l4_9': 10.0, \n",
"  'l4_10': 5.0, \n",
"  'l4_11': 0.0, \n",
"  'q1_n': 1.0, \n",
"  'q1_1': 10.0, \n",
"  'e1_n': 15.0, \n",
"  'e1_1': 9.0, \n",
"  'e1_2': 9.0, \n",
"  'e1_3': 10.0, \n",
"  'e1_4': 9.0, \n",
"  'e1_5': 7.0, \n",
"  'e1_6': 9.0, \n",
"  'e1_7': 0.0, \n",
"  'e1_8': 0.0, \n",
"  'e1_9': 10.0, \n",
"  'e1_10': 10.0, \n",
"  'e1_11': 9.0, \n",
"  'e1_12': 5.0, \n",
"  'e1_13': 10.0, \n",
"  'e1_14': 8.0, \n",
"  'e1_15': 0.0}, \n",
" {'l1_n': 1.0, \n",
"  'l1_1': 10.0, \n",
"  'l2_n': 7.0, \n",
"  'l2_1': 10.0, \n",
"  'l2_2': 10.0, \n",
"  'l2_3': 0.0, \n",
"  'l2_4': 5.0, \n",
"  'l2_5': 10.0, \n",
"  'l2_6': 10.0, \n",
"  'l2_7': 9.5, \n",
"  'l3_n': 14.0, \n",
"  'l3_1': 9.5, \n",
"  'l3_2': 10.0, \n",
"  'l3_3': 10.0, \n",
"  'l3_4': 8.0, \n",
"  'l3_5': 10.0, \n",
"  'l3_6': 8.0, \n",
"  'l3_7': 9.0, \n",
"  'l3_8': 0.0, \n",
"  'l3_9': 0.0, \n",
"  'l3_10': 0.0, \n",
"  'l3_11': 0.0, \n",
"  'l3_12': 0.0, \n",
"  'l3_13': 0.0, \n
```

```
"  'l3_l4': 0.0, \n",
"  'l4_n': 11.0, \n",
"  'l4_1': 0.0, \n",
"  'l4_2': 10.0, \n",
"  'l4_3': 10.0, \n",
"  'l4_4': 0.0, \n",
"  'l4_5': 0.0, \n",
"  'l4_6': 10.0, \n",
"  'l4_7': 0.0, \n",
"  'l4_8': 0.0, \n",
"  'l4_9': 0.0, \n",
"  'l4_10': 0.0, \n",
"  'l4_11': 0.0, \n",
"  'q1_n': 1.0, \n",
"  'q1_1': 10.0, \n",
"  'e1_n': 15.0, \n",
"  'e1_1': 9.0, \n",
"  'e1_2': 9.0, \n",
"  'e1_3': 10.0, \n",
"  'e1_4': 9.0, \n",
"  'e1_5': 0.0, \n",
"  'e1_6': 0.0, \n",
"  'e1_7': 0.0, \n",
"  'e1_8': 0.0, \n",
"  'e1_9': 0.0, \n",
"  'e1_10': 0.0, \n",
"  'e1_11': 0.0, \n",
"  'e1_12': 0.0, \n",
"  'e1_13': 0.0, \n",
"  'e1_14': 0.0, \n",
"  'e1_15': 0.0}, \n",
" {'l1_n': 1.0, \n",
"  'l1_1': 10.0, \n",
"  'l2_n': 7.0, \n",
"  'l2_1': 0.0, \n",
"  'l2_2': 0.0, \n",
"  'l2_3': 0.0, \n",
"  'l2_4': 0.0, \n",
"  'l2_5': 0.0, \n",
"  'l2_6': 0.0, \n",
"  'l2_7': 0.0, \n",
"  'l3_n': 14.0, \n",
"  'l3_1': 9.0, \n",
"  'l3_2': 10.0, \n",
"  'l3_3': 10.0, \n",
"  'l3_4': 10.0, \n",
"  'l3_5': 7.0, \n",
"  'l3_6': 10.0, \n",
"  'l3_7': 3.0, \n",
"  'l3_8': 6.0, \n",
"  'l3_9': 3.0, \n",
"  'l3_10': 3.0, \n",
"  'l3_11': 0.0, \n",
"  'l3_12': 0.0, \n
```

```

"  'l3_l3': 0.0,\n",
"  'l3_l4': 0.0,\n",
"  'l4_n': 11.0,\n",
"  'l4_1': 0.0,\n",
"  'l4_2': 0.0,\n",
"  'l4_3': 0.0,\n",
"  'l4_4': 0.0,\n",
"  'l4_5': 0.0,\n",
"  'l4_6': 0.0,\n",
"  'l4_7': 0.0,\n",
"  'l4_8': 0.0,\n",
"  'l4_9': 0.0,\n",
"  'l4_10': 0.0,\n",
"  'l4_11': 0.0,\n",
"  'q1_n': 1.0,\n",
"  'q1_1': 5.0,\n",
"  'e1_n': 15.0,\n",
"  'e1_1': 5.0,\n",
"  'e1_2': 5.0,\n",
"  'e1_3': 5.0,\n",
"  'e1_4': 5.0,\n",
"  'e1_5': 0.0,\n",
"  'e1_6': 0.0,\n",
"  'e1_7': 0.0,\n",
"  'e1_8': 0.0,\n",
"  'e1_9': 0.0,\n",
"  'e1_10': 0.0,\n",
"  'e1_11': 0.0,\n",
"  'e1_12': 0.0,\n",
"  'e1_13': 0.0,\n",
"  'e1_14': 0.0,\n",
"  'e1_15': 0.0},\n",
" {'l1_n': 1.0,\n",
"  'l1_1': 10.0,\n",
"  'l2_n': 7.0,\n",
"  'l2_1': 10.0,\n",
"  'l2_2': 10.0,\n",
"  'l2_3': 10.0,\n",
"  'l2_4': 9.5,\n",
"  'l2_5': 10.0,\n",
"  'l2_6': 10.0,\n",
"  'l2_7': 9.5,\n",
"  'l3_n': 14.0,\n",
"  'l3_1': 5.0,\n",
"  'l3_2': 9.5,\n",
"  'l3_3': 9.5,\n",
"  'l3_4': 8.0,\n",
"  'l3_5': 10.0,\n",
"  'l3_6': 10.0,\n",
"  'l3_7': 8.0,\n",
"  'l3_8': 10.0,\n",
"  'l3_9': 8.0,\n",
"  'l3_10': 0.0,\n",
"  'l3_11': 5.0,\n",

```

```

"  'l3_12': 6.0, \n",
"  'l3_13': 0.0, \n",
"  'l3_14': 0.0, \n",
"  'l4_n': 11.0, \n",
"  'l4_1': 0.0, \n",
"  'l4_2': 10.0, \n",
"  'l4_3': 10.0, \n",
"  'l4_4': 10.0, \n",
"  'l4_5': 0.0, \n",
"  'l4_6': 5.0, \n",
"  'l4_7': 0.0, \n",
"  'l4_8': 0.0, \n",
"  'l4_9': 0.0, \n",
"  'l4_10': 0.0, \n",
"  'l4_11': 0.0, \n",
"  'q1_n': 1.0, \n",
"  'q1_1': 9.5, \n",
"  'e1_n': 15.0, \n",
"  'e1_1': 9.0, \n",
"  'e1_2': 9.0, \n",
"  'e1_3': 10.0, \n",
"  'e1_4': 9.0, \n",
"  'e1_5': 9.0, \n",
"  'e1_6': 10.0, \n",
"  'e1_7': 7.0, \n",
"  'e1_8': 0.0, \n",
"  'e1_9': 9.0, \n",
"  'e1_10': 9.0, \n",
"  'e1_11': 9.0, \n",
"  'e1_12': 0.0, \n",
"  'e1_13': 5.0, \n",
"  'e1_14': 0.0, \n",
"  'e1_15': 0.0}, \n",
" {'l1_n': 1.0, \n",
"  'l1_1': 10.0, \n",
"  'l2_n': 7.0, \n",
"  'l2_1': 10.0, \n",
"  'l2_2': 10.0, \n",
"  'l2_3': 9.5, \n",
"  'l2_4': 0.0, \n",
"  'l2_5': 10.0, \n",
"  'l2_6': 10.0, \n",
"  'l2_7': 0.0, \n",
"  'l3_n': 14.0, \n",
"  'l3_1': 9.5, \n",
"  'l3_2': 10.0, \n",
"  'l3_3': 10.0, \n",
"  'l3_4': 10.0, \n",
"  'l3_5': 10.0, \n",
"  'l3_6': 10.0, \n",
"  'l3_7': 10.0, \n",
"  'l3_8': 10.0, \n",
"  'l3_9': 0.0, \n",
"  'l3_10': 0.0, \n",

```

```

"  'l3_l1': 10.0,\n",
"  'l3_l2': 5.0,\n",
"  'l3_l3': 10.0,\n",
"  'l3_l4': 10.0,\n",
"  'l4_n': 11.0,\n",
"  'l4_1': 0.0,\n",
"  'l4_2': 10.0,\n",
"  'l4_3': 10.0,\n",
"  'l4_4': 0.0,\n",
"  'l4_5': 0.0,\n",
"  'l4_6': 5.0,\n",
"  'l4_7': 0.0,\n",
"  'l4_8': 0.0,\n",
"  'l4_9': 0.0,\n",
"  'l4_10': 0.0,\n",
"  'l4_11': 0.0,\n",
"  'q1_n': 1.0,\n",
"  'q1_1': 0.0,\n",
"  'e1_n': 15.0,\n",
"  'e1_1': 9.0,\n",
"  'e1_2': 9.0,\n",
"  'e1_3': 10.0,\n",
"  'e1_4': 0.0,\n",
"  'e1_5': 8.0,\n",
"  'e1_6': 9.0,\n",
"  'e1_7': 7.0,\n",
"  'e1_8': 9.0,\n",
"  'e1_9': 10.0,\n",
"  'e1_10': 10.0,\n",
"  'e1_11': 10.0,\n",
"  'e1_12': 10.0,\n",
"  'e1_13': 10.0,\n",
"  'e1_14': 0.0,\n",
"  'e1_15': 0.0},\n",
" {'l1_n': 1.0,\n",
"  'l1_1': 10.0,\n",
"  'l2_n': 7.0,\n",
"  'l2_1': 0.0,\n",
"  'l2_2': 10.0,\n",
"  'l2_3': 10.0,\n",
"  'l2_4': 8.0,\n",
"  'l2_5': 10.0,\n",
"  'l2_6': 10.0,\n",
"  'l2_7': 10.0,\n",
"  'l3_n': 14.0,\n",
"  'l3_1': 9.0,\n",
"  'l3_2': 0.0,\n",
"  'l3_3': 0.0,\n",
"  'l3_4': 0.0,\n",
"  'l3_5': 0.0,\n",
"  'l3_6': 0.0,\n",
"  'l3_7': 0.0,\n",
"  'l3_8': 0.0,\n",
"  'l3_9': 0.0,\n",

```

```

"  '13_10': 0.0, \n",
"  '13_11': 0.0, \n",
"  '13_12': 0.0, \n",
"  '13_13': 0.0, \n",
"  '13_14': 0.0, \n",
"  '14_n': 11.0, \n",
"  '14_1': 0.0, \n",
"  '14_2': 0.0, \n",
"  '14_3': 0.0, \n",
"  '14_4': 0.0, \n",
"  '14_5': 0.0, \n",
"  '14_6': 0.0, \n",
"  '14_7': 0.0, \n",
"  '14_8': 0.0, \n",
"  '14_9': 0.0, \n",
"  '14_10': 0.0, \n",
"  '14_11': 0.0, \n",
"  'q1_n': 1.0, \n",
"  'q1_1': 9.5, \n",
"  'e1_n': 15.0, \n",
"  'e1_1': 9.0, \n",
"  'e1_2': 9.0, \n",
"  'e1_3': 0.0, \n",
"  'e1_4': 9.0, \n",
"  'e1_5': 8.0, \n",
"  'e1_6': 0.0, \n",
"  'e1_7': 0.0, \n",
"  'e1_8': 0.0, \n",
"  'e1_9': 0.0, \n",
"  'e1_10': 0.0, \n",
"  'e1_11': 0.0, \n",
"  'e1_12': 0.0, \n",
"  'e1_13': 0.0, \n",
"  'e1_14': 0.0, \n",
"  'e1_15': 0.0}, \n",
" {'11_n': 1.0, \n",
"  '11_1': 10.0, \n",
"  '12_n': 7.0, \n",
"  '12_1': 10.0, \n",
"  '12_2': 10.0, \n",
"  '12_3': 9.5, \n",
"  '12_4': 9.5, \n",
"  '12_5': 10.0, \n",
"  '12_6': 10.0, \n",
"  '12_7': 9.5, \n",
"  '13_n': 14.0, \n",
"  '13_1': 9.5, \n",
"  '13_2': 10.0, \n",
"  '13_3': 10.0, \n",
"  '13_4': 10.0, \n",
"  '13_5': 8.0, \n",
"  '13_6': 10.0, \n",
"  '13_7': 8.0, \n",
"  '13_8': 10.0, \n",

```

```
"  '13_9': 10.0, \n",
"  '13_10': 7.0, \n",
"  '13_11': 5.0, \n",
"  '13_12': 0.0, \n",
"  '13_13': 0.0, \n",
"  '13_14': 0.0, \n",
"  '14_n': 11.0, \n",
"  '14_1': 10.0, \n",
"  '14_2': 10.0, \n",
"  '14_3': 10.0, \n",
"  '14_4': 10.0, \n",
"  '14_5': 5.0, \n",
"  '14_6': 6.0, \n",
"  '14_7': 0.0, \n",
"  '14_8': 0.0, \n",
"  '14_9': 0.0, \n",
"  '14_10': 0.0, \n",
"  '14_11': 0.0, \n",
"  'q1_n': 1.0, \n",
"  'q1_1': 10.0, \n",
"  'e1_n': 15.0, \n",
"  'e1_1': 9.0, \n",
"  'e1_2': 9.0, \n",
"  'e1_3': 10.0, \n",
"  'e1_4': 9.0, \n",
"  'e1_5': 8.0, \n",
"  'e1_6': 9.0, \n",
"  'e1_7': 7.0, \n",
"  'e1_8': 9.0, \n",
"  'e1_9': 10.0, \n",
"  'e1_10': 10.0, \n",
"  'e1_11': 10.0, \n",
"  'e1_12': 10.0, \n",
"  'e1_13': 0.0, \n",
"  'e1_14': 0.0, \n",
"  'e1_15': 0.0}, \n",
" {'11_n': 1.0, \n",
"  '11_1': 10.0, \n",
"  '12_n': 7.0, \n",
"  '12_1': 10.0, \n",
"  '12_2': 10.0, \n",
"  '12_3': 0.0, \n",
"  '12_4': 0.0, \n",
"  '12_5': 10.0, \n",
"  '12_6': 10.0, \n",
"  '12_7': 7.0, \n",
"  '13_n': 14.0, \n",
"  '13_1': 10.0, \n",
"  '13_2': 10.0, \n",
"  '13_3': 10.0, \n",
"  '13_4': 10.0, \n",
"  '13_5': 7.0, \n",
"  '13_6': 10.0, \n",
"  '13_7': 6.0, \n",
```



```
"  '13_8': 3.0,\n",  
"  '13_9': 10.0,\n",  
"  '13_10': 10.0,\n",  
"  '13_11': 10.0,\n",  
"  '13_12': 10.0,\n",  
"  '13_13': 10.0,\n",  
"  '13_14': 10.0,\n",  
"  '14_n': 11.0,\n",  
"  '14_1': 10.0,\n",  
"  '14_2': 10.0,\n",  
"  '14_3': 10.0,\n",  
"  '14_4': 10.0,\n",  
"  '14_5': 10.0,\n",  
"  '14_6': 5.0,\n",  
"  '14_7': 10.0,\n",  
"  '14_8': 10.0,\n",  
"  '14_9': 10.0,\n",  
"  '14_10': 10.0,\n",  
"  '14_11': 10.0,\n",  
"  'q1_n': 1.0,\n",  
"  'q1_1': 0.0,\n",  
"  'e1_n': 15.0,\n",  
"  'e1_1': 9.0,\n",  
"  'e1_2': 9.0,\n",  
"  'e1_3': 9.0,\n",  
"  'e1_4': 9.0,\n",  
"  'e1_5': 9.0,\n",  
"  'e1_6': 10.0,\n",  
"  'e1_7': 9.0,\n",  
"  'e1_8': 9.0,\n",  
"  'e1_9': 10.0,\n",  
"  'e1_10': 10.0,\n",  
"  'e1_11': 10.0,\n",  
"  'e1_12': 10.0,\n",  
"  'e1_13': 10.0,\n",  
"  'e1_14': 5.0,\n",  
"  'e1_15': 10.0},\n",  
"  {'11_n': 1.0,\n",  
"  '11_1': 10.0,\n",  
"  '12_n': 7.0,\n",  
"  '12_1': 10.0,\n",  
"  '12_2': 10.0,\n",  
"  '12_3': 9.5,\n",  
"  '12_4': 0.0,\n",  
"  '12_5': 10.0,\n",  
"  '12_6': 10.0,\n",  
"  '12_7': 0.0,\n",  
"  '13_n': 14.0,\n",  
"  '13_1': 9.5,\n",  
"  '13_2': 0.0,\n",  
"  '13_3': 0.0,\n",  
"  '13_4': 10.0,\n",  
"  '13_5': 0.0,\n",  
"  '13_6': 10.0,\n",
```

```
" '13_7': 5.0,\n",  
" '13_8': 10.0,\n",  
" '13_9': 7.0,\n",  
" '13_10': 0.0,\n",  
" '13_11': 10.0,\n",  
" '13_12': 6.0,\n",  
" '13_13': 10.0,\n",  
" '13_14': 0.0,\n",  
" '14_n': 11.0,\n",  
" '14_1': 10.0,\n",  
" '14_2': 10.0,\n",  
" '14_3': 6.0,\n",  
" '14_4': 0.0,\n",  
" '14_5': 0.0,\n",  
" '14_6': 0.0,\n",  
" '14_7': 0.0,\n",  
" '14_8': 0.0,\n",  
" '14_9': 0.0,\n",  
" '14_10': 0.0,\n",  
" '14_11': 0.0,\n",  
" 'q1_n': 1.0,\n",  
" 'q1_1': 0.0,\n",  
" 'e1_n': 15.0,\n",  
" 'e1_1': 0.0,\n",  
" 'e1_2': 0.0,\n",  
" 'e1_3': 0.0,\n",  
" 'e1_4': 0.0,\n",  
" 'e1_5': 5.0,\n",  
" 'e1_6': 0.0,\n",  
" 'e1_7': 7.0,\n",  
" 'e1_8': 0.0,\n",  
" 'e1_9': 3.0,\n",  
" 'e1_10': 3.0,\n",  
" 'e1_11': 3.0,\n",  
" 'e1_12': 0.0,\n",  
" 'e1_13': 3.0,\n",  
" 'e1_14': 0.0,\n",  
" 'e1_15': 0.0},\n",  
" {'11_n': 1.0,\n",  
" '11_1': 10.0,\n",  
" '12_n': 7.0,\n",  
" '12_1': 0.0,\n",  
" '12_2': 0.0,\n",  
" '12_3': 0.0,\n",  
" '12_4': 0.0,\n",  
" '12_5': 0.0,\n",  
" '12_6': 0.0,\n",  
" '12_7': 0.0,\n",  
" '13_n': 14.0,\n",  
" '13_1': 0.0,\n",  
" '13_2': 0.0,\n",  
" '13_3': 0.0,\n",  
" '13_4': 0.0,\n",  
" '13_5': 0.0,\n",
```

```

    " 'l3_6': 0.0,\n",
    " 'l3_7': 0.0,\n",
    " 'l3_8': 0.0,\n",
    " 'l3_9': 0.0,\n",
    " 'l3_10': 0.0,\n",
    " 'l3_11': 0.0,\n",
    " 'l3_12': 0.0,\n",
    " 'l3_13': 0.0,\n",
    " 'l3_14': 0.0,\n",
    " 'l4_n': 11.0,\n",
    " 'l4_1': 0.0,\n",
    " 'l4_2': 0.0,\n",
    " 'l4_3': 0.0,\n",
    " 'l4_4': 0.0,\n",
    " 'l4_5': 0.0,\n",
    " 'l4_6': 0.0,\n",
    " 'l4_7': 0.0,\n",
    " 'l4_8': 0.0,\n",
    " 'l4_9': 0.0,\n",
    " 'l4_10': 0.0,\n",
    " 'l4_11': 0.0,\n",
    " 'q1_n': 1.0,\n",
    " 'q1_1': 0.0,\n",
    " 'e1_n': 15.0,\n",
    " 'e1_1': 0.0,\n",
    " 'e1_2': 0.0,\n",
    " 'e1_3': 0.0,\n",
    " 'e1_4': 0.0,\n",
    " 'e1_5': 0.0,\n",
    " 'e1_6': 0.0,\n",
    " 'e1_7': 0.0,\n",
    " 'e1_8': 0.0,\n",
    " 'e1_9': 0.0,\n",
    " 'e1_10': 0.0,\n",
    " 'e1_11': 0.0,\n",
    " 'e1_12': 0.0,\n",
    " 'e1_13': 0.0,\n",
    " 'e1_14': 0.0,\n",
    " 'e1_15': 0.0}]"
  ]
},
"execution_count": 14,
"metadata": {},
"output_type": "execute_result"
}
],
"source": [
  "new_data"
]
},
{
  "cell_type": "code",
  "execution_count": 15,
  "metadata": {},

```

```

"outputs": [
  {
    "data": {
      "text/plain": [
        "10.0"
      ]
    },
    "execution_count": 15,
    "metadata": {},
    "output_type": "execute_result"
  }
],
"source": [
  "new_data[10][\"13_4\"]"
]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "### Data Representation in Custom Class\n",
    "\n",
    "Our representation of a table as a list of dictionaries isn't the  

    most efficient, but it is convenient. \n",
    "\n",
    "Lets try a different approach to storing the data that is custom  

    made for storing tables with rows of data. \n"
  ]
},
{
  "cell_type": "code",
  "execution_count": 16,
  "metadata": {},
  "outputs": [],
  "source": [
    "class DataRow:\n",
    "    def __init__(self, fields, data):\n",
    "        self.__fields=fields\n",
    "        self.__data=data\n",
    "        \n",
    "    def __getitem__(self, key):\n",
    "        return self.__data[self.__fields.index(key)]\n",
    "    \n",
    "    \n",
    "class Data:\n",
    "    def __init__(self):\n",
    "        self.__fields=list()\n",
    "        self.__data=list()\n",
    "        \n",
    "    def set_fields(self, fields):\n",
    "        self.__fields=fields\n",
    "        \n",
    "    def add_data_point(self, data_point):\n",
    "        if isinstance(data_point, list):\n",

```

```

        "            if len(data_point) == len(self.__fields):\n",
        "
self.__data.append(DataRow(self.__fields,data_point))\n",
        "            else:\n",
        "                print(\"Expected {} fields, got {}
fields.\").format(len(self.__fields),len(fields))\n",
        "            else:\n",
        "                print(\"Data Point must be given as a list.\")\n",
        "\n",
        "    def add_data_points(self,data_points):\n",
        "        for data_point in data_points:\n",
        "            self.add_data_point(data_point)\n",
        "        \n",
        "    def fields(self):\n",
        "        return self.__fields\n",
        "    \n",
        "    def __getitem__(self,key):\n",
        "        return self.__data[key]\n",
        "\n",
        "    def __str__(self):\n",
        "        return self.__fields"
]
},
{
    "cell_type": "code",
    "execution_count": 17,
    "metadata": {},
    "outputs": [],
    "source": [
        "my_data=Data()\n",
        "my_data.set_fields(fields)\n",
        "my_data.add_data_points(data)"
    ]
},
{
    "cell_type": "code",
    "execution_count": 18,
    "metadata": {},
    "outputs": [
        {
            "data": {
                "text/plain": [
                    "10.0"
                ]
            },
            "execution_count": 18,
            "metadata": {},
            "output_type": "execute_result"
        }
    ],
    "source": [
        "my_data[12][\"13_4\"]"
    ]
},

```

```

{
  "cell_type": "code",
  "execution_count": 19,
  "metadata": {},
  "outputs": [
    {
      "data": {
        "text/plain": [
          "<__main__.DataRow at 0x10397f610>"
        ]
      },
      "execution_count": 19,
      "metadata": {},
      "output_type": "execute_result"
    }
  ],
  "source": [
    "my_data[12]"
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "Now lets put it all together:"
  ]
},
{
  "cell_type": "code",
  "execution_count": 22,
  "metadata": {},
  "outputs": [],
  "source": [
    "class DataFileHandler:\n",
    "    def __init__(self,extensions):\n",
    "        self.__extensions=extensions\n",
    "        \n",
    "    def check_extension(self,filename):\n",
    "        file_extension=filename.split(\".\")[-1]\n",
    "        return file_extension in self.__extensions\n",
    "    \n",
    "    def _readfile(self,filename):\n",
    "        raise NotImplementedError\n",
    "    \n",
    "    def readfile(self,filename,check_extension=True):\n",
    "        if not check_extension or\nself.check_extension(filename):\n",
    "            return self._readfile(filename)\n",
    "        else:\n",
    "            print(\"Error: filename {} does not match acceptable\n",
    "            extensions.\".format(filename))\n",
    "    \n",
    "    def _writefile(self,filename,data):\n",
    "        raise NotImplementedError\n",

```

```

        \n",
        def writefile(self,filename,data):\n",
        return self._writefile(filename,data)\n",
        \n",
        \n",
"class CSVHandler(DataFileHandler):\n",
    def __init__(self):\n",
        #super(CSVHandler,self).__init__([\\"csv\\",\\"CSV\\"])\n",
        DataFileHandler.__init__(self,[\\"csv\\",\\"CSV\\"])\n",
        \n",
    def _readfile(self,filename):\n",
        f=open(filename,\\r\\")\n",
        first_line = f.readline().rstrip()\n",
        fields=first_line.split(\\",\\")\n",
\n",
        data=list()\n",
\n",
        line = f.readline().rstrip()\n",
        while line:\n",
            items=line.split(\\",\\")\n",
            \n",
            row=list()\n",
            for item in items:\n",
                try:\n",
                    d=float(item)\n",
                except ValueError:\n",
                    d=item\n",
                row.append(d)\n",
            \n",
            data.append(row)\n",
            \n",
            line = f.readline().rstrip()\n",
\n",
        f.close()\n",
        \n",
        my_data=Data()\n",
        my_data.set_fields(fields)\n",
        my_data.add_data_points(data)\n",
        \n",
        return my_data\n",
        \n",
        "
    ]
},
{
    "cell_type": "code",
    "execution_count": 23,
    "metadata": {},
    "outputs": [],
    "source": [
        "my_handler=CSVHandler()\n",
        "my_data=my_handler.readfile(\\\"Scores.csv\\")"
    ]
},

```

```

{
  "cell_type": "code",
  "execution_count": 24,
  "metadata": {},
  "outputs": [
    {
      "data": {
        "text/plain": [
          "0.0"
        ]
      },
      "execution_count": 24,
      "metadata": {},
      "output_type": "execute_result"
    }
  ],
  "source": [
    "my_data[10][\"l4_1\"]"
  ]
},
{
  "cell_type": "code",
  "execution_count": 25,
  "metadata": {},
  "outputs": [
    {
      "data": {
        "text/plain": [
          "__main__.Data"
        ]
      },
      "execution_count": 25,
      "metadata": {},
      "output_type": "execute_result"
    }
  ],
  "source": [
    "type(my_data)"
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "## Pandas\n",
    "\n",
    "What we just build is very similar to Pandas..."
  ]
},
{
  "cell_type": "code",
  "execution_count": 26,
  "metadata": {},
  "outputs": [],

```



```

"source": [
  "import pandas as pd\n",
  "Data=pd.read_csv(\"Scores.csv\") "
]
},
{
  "cell_type": "code",
  "execution_count": 27,
  "metadata": {},
  "outputs": [
    {
      "data": {
        "text/plain": [
          "[ 'read_clipboard',\n",
          " 'read_csv',\n",
          " 'read_excel',\n",
          " 'read_feather',\n",
          " 'read_fwf',\n",
          " 'read_gbq',\n",
          " 'read_hdf',\n",
          " 'read_html',\n",
          " 'read_json',\n",
          " 'read_orc',\n",
          " 'read_parquet',\n",
          " 'read_pickle',\n",
          " 'read_sas',\n",
          " 'read_spss',\n",
          " 'read_sql',\n",
          " 'read_sql_query',\n",
          " 'read_sql_table',\n",
          " 'read_stata',\n",
          " 'read_table',\n",
          " 'read_xml']"
        ]
      },
      "execution_count": 27,
      "metadata": {},
      "output_type": "execute_result"
    }
  ],
  "source": [
    "list(filter(lambda x: \"read\" in x, dir(pd)))"
  ]
},
{
  "cell_type": "code",
  "execution_count": 28,
  "metadata": {},
  "outputs": [
    {
      "data": {
        "text/plain": [
          "pandas.core.frame.DataFrame"
        ]
      }
    }
  ]
}

```

```

    },
    "execution_count": 28,
    "metadata": {},
    "output_type": "execute_result"
  }
],
"source": [
  "type(Data) "
]
},
{
  "cell_type": "code",
  "execution_count": 29,
  "metadata": {
    "scrolled": true
  },
  "outputs": [
    {
      "data": {
        "text/html": [
          "<div>\n",
          "<style scoped>\n",
          "    .dataframe tbody tr th:only-of-type {\n",
          "        vertical-align: middle;\n",
          "    }\n",
          "\n",
          "    .dataframe tbody tr th {\n",
          "        vertical-align: top;\n",
          "    }\n",
          "\n",
          "    .dataframe thead th {\n",
          "        text-align: right;\n",
          "    }\n",
          "</style>\n",
          "<table border=\"1\" class=\"dataframe\">\n",
          "  <thead>\n",
          "    <tr style=\"text-align: right;\">\n",
          "      <th></th>\n",
          "      <th>l1_n</th>\n",
          "      <th>l1_1</th>\n",
          "      <th>l2_n</th>\n",
          "      <th>l2_1</th>\n",
          "      <th>l2_2</th>\n",
          "      <th>l2_3</th>\n",
          "      <th>l2_4</th>\n",
          "      <th>l2_5</th>\n",
          "      <th>l2_6</th>\n",
          "      <th>l2_7</th>\n",
          "      <th>...</th>\n",
          "      <th>e1_6</th>\n",
          "      <th>e1_7</th>\n",
          "      <th>e1_8</th>\n",
          "      <th>e1_9</th>\n",
          "      <th>e1_10</th>\n",

```

```

"         <th>e1_11</th>\n",
"         <th>e1_12</th>\n",
"         <th>e1_13</th>\n",
"         <th>e1_14</th>\n",
"         <th>e1_15</th>\n",
"     </tr>\n",
" </thead>\n",
" <tbody>\n",
"     <tr>\n",
"         <th>0</th>\n",
"         <td>1</td>\n",
"         <td>10</td>\n",
"         <td>7</td>\n",
"         <td>10</td>\n",
"         <td>10</td>\n",
"         <td>9.5</td>\n",
"         <td>9.0</td>\n",
"         <td>10</td>\n",
"         <td>10</td>\n",
"         <td>9.5</td>\n",
"         <td>...</td>\n",
"         <td>0</td>\n",
"         <td>0</td>\n",
"         <td>10</td>\n",
"         <td>10</td>\n",
"         <td>10</td>\n",
"         <td>10</td>\n",
"         <td>0</td>\n",
"         <td>10</td>\n",
"         <td>5</td>\n",
"         <td>10</td>\n",
"     </tr>\n",
"     <tr>\n",
"         <th>1</th>\n",
"         <td>1</td>\n",
"         <td>10</td>\n",
"         <td>7</td>\n",
"         <td>0</td>\n",
"         <td>10</td>\n",
"         <td>9.5</td>\n",
"         <td>0.0</td>\n",
"         <td>10</td>\n",
"         <td>10</td>\n",
"         <td>0.0</td>\n",
"         <td>...</td>\n",
"         <td>0</td>\n",
"         <td>9</td>\n",
"         <td>9</td>\n",
"         <td>9</td>\n",
"         <td>9</td>\n",
"         <td>10</td>\n",
"         <td>5</td>\n",
"         <td>0</td>\n",

```

```

"      <td>0</td>\n",
"    </tr>\n",
"  <tr>\n",
"    <th>2</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",
"    <td>7</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>0.0</td>\n",
"    <td>10.0</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10.0</td>\n",
"    <td>...</td>\n",
"    <td>9</td>\n",
"    <td>7</td>\n",
"    <td>9</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>5</td>\n",
"    <td>10</td>\n",
"    <td>5</td>\n",
"    <td>0</td>\n",
"  </tr>\n",
"  <tr>\n",
"    <th>3</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",
"    <td>7</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10.0</td>\n",
"    <td>9.5</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>9.5</td>\n",
"    <td>...</td>\n",
"    <td>0</td>\n",
"    <td>7</td>\n",
"    <td>5</td>\n",
"    <td>9</td>\n",
"    <td>9</td>\n",
"    <td>9</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"  </tr>\n",
"  <tr>\n",
"    <th>4</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",

```

```

"      <td>7</td>\n",
"      <td>10</td>\n",
"      <td>10</td>\n",
"      <td>5.0</td>\n",
"      <td>9.5</td>\n",
"      <td>10</td>\n",
"      <td>10</td>\n",
"      <td>9.5</td>\n",
"      <td>...</td>\n",
"      <td>10</td>\n",
"      <td>0</td>\n",
"      <td>9</td>\n",
"      <td>10</td>\n",
"      <td>9</td>\n",
"      <td>10</td>\n",
"      <td>9</td>\n",
"      <td>5</td>\n",
"      <td>0</td>\n",
"      <td>0</td>\n",
"    </tr>\n",
"  <tr>\n",
"    <th>5</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",
"    <td>7</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>3.0</td>\n",
"    <td>7.0</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>9.0</td>\n",
"    <td>...</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>9</td>\n",
"    <td>8</td>\n",
"    <td>2</td>\n",
"  </tr>\n",
"  <tr>\n",
"    <th>6</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",
"    <td>7</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>3.0</td>\n",
"    <td>9.5</td>\n",
"    <td>10</td>\n",

```

```

"      <td>10</td>\n",
"      <td>9.5</td>\n",
"      <td>...</td>\n",
"      <td>9</td>\n",
"      <td>0</td>\n",
"      <td>0</td>\n",
"      <td>10</td>\n",
"      <td>10</td>\n",
"      <td>9</td>\n",
"      <td>5</td>\n",
"      <td>10</td>\n",
"      <td>8</td>\n",
"      <td>0</td>\n",
"    </tr>\n",
"  <tr>\n",
"    <th>7</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",
"    <td>7</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>0.0</td>\n",
"    <td>5.0</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>9.5</td>\n",
"    <td>...</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"  </tr>\n",
"  <tr>\n",
"    <th>8</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",
"    <td>7</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0.0</td>\n",
"    <td>0.0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0.0</td>\n",
"    <td>...</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>

```

[illegible]

```

"      <td>0</td>\n",
"    </tr>\n",
"  <tr>\n",
"    <th>11</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",
"    <td>7</td>\n",
"    <td>0</td>\n",
"    <td>10</td>\n",
"    <td>10.0</td>\n",
"    <td>8.0</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10.0</td>\n",
"    <td>...</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"  </tr>\n",
"  <tr>\n",
"    <th>12</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",
"    <td>7</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>9.5</td>\n",
"    <td>9.5</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>9.5</td>\n",
"    <td>...</td>\n",
"    <td>9</td>\n",
"    <td>7</td>\n",
"    <td>9</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"  </tr>\n",
"  <tr>\n",
"    <th>13</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",

```



```

"      <td>7</td>\n",
"      <td>10</td>\n",
"      <td>10</td>\n",
"      <td>0.0</td>\n",
"      <td>0.0</td>\n",
"      <td>10</td>\n",
"      <td>10</td>\n",
"      <td>7.0</td>\n",
"      <td>...</td>\n",
"      <td>10</td>\n",
"      <td>9</td>\n",
"      <td>9</td>\n",
"      <td>10</td>\n",
"      <td>10</td>\n",
"      <td>10</td>\n",
"      <td>10</td>\n",
"      <td>10</td>\n",
"      <td>5</td>\n",
"      <td>10</td>\n",
"    </tr>\n",
"  <tr>\n",
"    <th>14</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",
"    <td>7</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>9.5</td>\n",
"    <td>0.0</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>0.0</td>\n",
"    <td>...</td>\n",
"    <td>0</td>\n",
"    <td>7</td>\n",
"    <td>0</td>\n",
"    <td>3</td>\n",
"    <td>3</td>\n",
"    <td>3</td>\n",
"    <td>0</td>\n",
"    <td>3</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"  </tr>\n",
"  <tr>\n",
"    <th>15</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",
"    <td>7</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0.0</td>\n",
"    <td>0.0</td>\n",
"    <td>0</td>\n",

```

```

"      <td>0</td>\n",
"      <td>0.0</td>\n",
"      <td>...</td>\n",
"      <td>0</td>\n",
"      <td>0</td>\n",
"      <td>0</td>\n",
"      <td>0</td>\n",
"      <td>0</td>\n",
"      <td>0</td>\n",
"      <td>0</td>\n",
"      <td>0</td>\n",
"      <td>0</td>\n",
"      <td>0</td>\n",
"      <td>0</td>\n",
"    </tr>\n",
"  </tbody>\n",
"</table>\n",
"<p>16 rows Ã– 55 columns</p>\n",
"</div>"
],
"text/plain": [
...   e1_6  11_n  11_1  12_n  12_1  12_2  12_3  12_4  12_5  12_6  12_7
"0      1    10     7    10    10    9.5    9.0    10    10    9.5
...   0    \n",
"1      1    10     7     0    10    9.5    0.0    10    10    0.0
...   0    \n",
"2      1    10     7    10    10    0.0   10.0    10    10   10.0
...   9    \n",
"3      1    10     7    10    10   10.0    9.5    10    10    9.5
...   0    \n",
"4      1    10     7    10    10    5.0    9.5    10    10    9.5
...  10    \n",
"5      1    10     7    10    10    3.0    7.0    10    10    9.0
...  10    \n",
"6      1    10     7    10    10    3.0    9.5    10    10    9.5
...   9    \n",
"7      1    10     7    10    10    0.0    5.0    10    10    9.5
...   0    \n",
"8      1    10     7     0     0    0.0    0.0     0     0    0.0
...   0    \n",
"9      1    10     7    10    10   10.0    9.5    10    10    9.5
...  10    \n",
"10     1    10     7    10    10    9.5    0.0    10    10    0.0
...   9    \n",
"11     1    10     7     0    10   10.0    8.0    10    10   10.0
...   0    \n",
"12     1    10     7    10    10    9.5    9.5    10    10    9.5
...   9    \n",
"13     1    10     7    10    10    0.0    0.0    10    10    7.0
...  10    \n",
"14     1    10     7    10    10    9.5    0.0    10    10    0.0
...   0    \n",
"15     1    10     7     0     0    0.0    0.0     0     0    0.0
...   0    \n",

```

```

        "\n",
        "      e1_7  e1_8  e1_9  e1_10  e1_11  e1_12  e1_13  e1_14  e1_15
\n",
        "0      0    10    10     10     10     0     10     5     10
\n",
        "1      9     9     9     9     9    10     5     0     0
\n",
        "2      7     9    10    10    10     5    10     5     0
\n",
        "3      7     5     9     9     9     0     0     0     0
\n",
        "4      0     9    10     9    10     9     5     0     0
\n",
        "5     10    10    10    10    10    10     9     8     2
\n",
        "6      0     0    10    10     9     5    10     8     0
\n",
        "7      0     0     0     0     0     0     0     0     0
\n",
        "8      0     0     0     0     0     0     0     0     0
\n",
        "9      7     0     9     9     9     0     5     0     0
\n",
        "10     7     9    10    10    10    10    10     0     0
\n",
        "11     0     0     0     0     0     0     0     0     0
\n",
        "12     7     9    10    10    10    10     0     0     0
\n",
        "13     9     9    10    10    10    10    10     5    10
\n",
        "14     7     0     3     3     3     0     3     0     0
\n",
        "15     0     0     0     0     0     0     0     0     0
\n",

```

```

        "\n",
        "[16 rows x 55 columns]"
    ]
},
"execution_count": 29,
"metadata": {},
"output_type": "execute_result"
}
],
"source": [
    "Data"
]
},
{
    "cell_type": "code",
    "execution_count": 30,
    "metadata": {},
    "outputs": [
        {

```

```

"data": {
  "text/plain": [
    "[ 'T',\n",
    " ' _AXIS_LEN',\n",
    " ' _AXIS_ORDERS',\n",
    " ' _AXIS_TO_AXIS_NUMBER',\n",
    " ' _HANDLED_TYPES',\n",
    " ' __abs__',\n",
    " ' __add__',\n",
    " ' __and__',\n",
    " ' __annotations__',\n",
    " ' __array__',\n",
    " ' __array_priority__',\n",
    " ' __array_ufunc__',\n",
    " ' __array_wrap__',\n",
    " ' __bool__',\n",
    " ' __class__',\n",
    " ' __contains__',\n",
    " ' __copy__',\n",
    " ' __dataframe__',\n",
    " ' __deepcopy__',\n",
    " ' __delattr__',\n",
    " ' __delitem__',\n",
    " ' __dict__',\n",
    " ' __dir__',\n",
    " ' __divmod__',\n",
    " ' __doc__',\n",
    " ' __eq__',\n",
    " ' __finalize__',\n",
    " ' __floordiv__',\n",
    " ' __format__',\n",
    " ' __ge__',\n",
    " ' __getattr__',\n",
    " ' __getattribute__',\n",
    " ' __getitem__',\n",
    " ' __getstate__',\n",
    " ' __gt__',\n",
    " ' __hash__',\n",
    " ' __iadd__',\n",
    " ' __iand__',\n",
    " ' __ifloordiv__',\n",
    " ' __imod__',\n",
    " ' __imul__',\n",
    " ' __init__',\n",
    " ' __init_subclass__',\n",
    " ' __invert__',\n",
    " ' __ior__',\n",
    " ' __ipow__',\n",
    " ' __isub__',\n",
    " ' __iter__',\n",
    " ' __itruediv__',\n",
    " ' __ixor__',\n",
    " ' __le__',\n",
    " ' __len__',\n
```

```
" ' __lt__ ',\n",
" ' __matmul__ ',\n",
" ' __mod__ ',\n",
" ' __module__ ',\n",
" ' __mul__ ',\n",
" ' __ne__ ',\n",
" ' __neg__ ',\n",
" ' __new__ ',\n",
" ' __nonzero__ ',\n",
" ' __or__ ',\n",
" ' __pos__ ',\n",
" ' __pow__ ',\n",
" ' __radd__ ',\n",
" ' __rand__ ',\n",
" ' __rdivmod__ ',\n",
" ' __reduce__ ',\n",
" ' __reduce_ex__ ',\n",
" ' __repr__ ',\n",
" ' __rfloordiv__ ',\n",
" ' __rmatmul__ ',\n",
" ' __rmod__ ',\n",
" ' __rmul__ ',\n",
" ' __ror__ ',\n",
" ' __round__ ',\n",
" ' __rpow__ ',\n",
" ' __rsub__ ',\n",
" ' __rtruediv__ ',\n",
" ' __rxor__ ',\n",
" ' __setattr__ ',\n",
" ' __setitem__ ',\n",
" ' __setstate__ ',\n",
" ' __sizeof__ ',\n",
" ' __str__ ',\n",
" ' __sub__ ',\n",
" ' __subclasshook__ ',\n",
" ' __truediv__ ',\n",
" ' __weakref__ ',\n",
" ' __xor__ ',\n",
" ' _accessors',\n",
" ' _accum_func',\n",
" ' _add_numeric_operations',\n",
" ' _agg_by_level',\n",
" ' _agg_examples_doc',\n",
" ' _agg_summary_and_see_also_doc',\n",
" ' _align_frame',\n",
" ' _align_series',\n",
" ' _append',\n",
" ' _arith_method',\n",
" ' _as_manager',\n",
" ' _attrs',\n",
" ' _box_col_values',\n",
" ' _can_fast_transpose',\n",
" ' _check_inplace_and_allows_duplicate_labels',\n",
" ' _check_inplace_setting',
```

```
" '_check_is_chained_assignment_possible',\n",  
" '_check_label_or_level_ambiguity',\n",  
" '_check_setitem_copy',\n",  
" '_clear_item_cache',\n",  
" '_clip_with_one_bound',\n",  
" '_clip_with_scalar',\n",  
" '_cmp_method',\n",  
" '_combine_frame',\n",  
" '_consolidate',\n",  
" '_consolidate_inplace',\n",  
" '_construct_axes_dict',\n",  
" '_construct_axes_from_arguments',\n",  
" '_construct_result',\n",  
" '_constructor',\n",  
" '_constructor_sliced',\n",  
" '_convert',\n",  
" '_count_level',\n",  
" '_data',\n",  
" '_dir_additions',\n",  
" '_dir_deletions',\n",  
" '_dispatch_frame_op',\n",  
" '_drop_axis',\n",  
" '_drop_labels_or_levels',\n",  
" '_ensure_valid_index',\n",  
" '_find_valid_index',\n",  
" '_flags',\n",  
" '_from_arrays',\n",  
" '_get_agg_axis',\n",  
" '_get_axis',\n",  
" '_get_axis_name',\n",  
" '_get_axis_number',\n",  
" '_get_axis_resolvers',\n",  
" '_get_block_manager_axis',\n",  
" '_get_bool_data',\n",  
" '_get_cleaned_column_resolvers',\n",  
" '_get_column_array',\n",  
" '_get_index_resolvers',\n",  
" '_get_item_cache',\n",  
" '_get_label_or_level_values',\n",  
" '_get_numeric_data',\n",  
" '_get_value',\n",  
" '_getitem_bool_array',\n",  
" '_getitem_multilevel',\n",  
" '_gotitem',\n",  
" '_hidden_attrs',\n",  
" '_indexed_same',\n",  
" '_info_axis',\n",  
" '_info_axis_name',\n",  
" '_info_axis_number',\n",  
" '_info_repr',\n",  
" '_init_mgr',\n",  
" '_inplace_method',\n",  
" '_internal_names',\n",  
" '_internal_names_set',\n",
```

```
" '_is_copy',\n",
" '_is_homogeneous_type',\n",
" '_is_label_or_level_reference',\n",
" '_is_label_reference',\n",
" '_is_level_reference',\n",
" '_is_mixed_type',\n",
" '_is_view',\n",
" '_iset_item',\n",
" '_iset_item_mgr',\n",
" '_iset_not_inplace',\n",
" '_item_cache',\n",
" '_iter_column_arrays',\n",
" '_ixs',\n",
" '_join_compat',\n",
" '_logical_func',\n",
" '_logical_method',\n",
" '_maybe_cache_changed',\n",
" '_maybe_update_cacher',\n",
" '_metadata',\n",
" '_mgr',\n",
" '_min_count_stat_function',\n",
" '_needs_reindex_multi',\n",
" '_protect_consolidate',\n",
" '_reduce',\n",
" '_reduce_axis1',\n",
" '_reindex_axes',\n",
" '_reindex_columns',\n",
" '_reindex_index',\n",
" '_reindex_multi',\n",
" '_reindex_with_indexers',\n",
" '_rename',\n",
" '_replace_columnwise',\n",
" '_repr_data_resource_',\n",
" '_repr_fits_horizontal_',\n",
" '_repr_fits_vertical_',\n",
" '_repr_html_',\n",
" '_repr_latex_',\n",
" '_reset_cache',\n",
" '_reset_cacher',\n",
" '_sanitize_column',\n",
" '_series',\n",
" '_set_axis',\n",
" '_set_axis_name',\n",
" '_set_axis_nocheck',\n",
" '_set_is_copy',\n",
" '_set_item',\n",
" '_set_item_frame_value',\n",
" '_set_item_mgr',\n",
" '_set_value',\n",
" '_setitem_array',\n",
" '_setitem_frame',\n",
" '_setitem_slice',\n",
" '_slice',\n",
" '_stat_axis',
```

```
" '_stat_axis_name',\n",
" '_stat_axis_number',\n",
" '_stat_function',\n",
" '_stat_function_ddof',\n",
" '_take',\n",
" '_take_with_is_copy',\n",
" '_to_dict_of_blocks',\n",
" '_typ',\n",
" '_update_inplace',\n",
" '_validate_dtype',\n",
" '_values',\n",
" '_where',\n",
" 'abs',\n",
" 'add',\n",
" 'add_prefix',\n",
" 'add_suffix',\n",
" 'agg',\n",
" 'aggregate',\n",
" 'align',\n",
" 'all',\n",
" 'any',\n",
" 'append',\n",
" 'apply',\n",
" 'applymap',\n",
" 'asfreq',\n",
" 'asof',\n",
" 'assign',\n",
" 'astype',\n",
" 'at',\n",
" 'at_time',\n",
" 'attrs',\n",
" 'axes',\n",
" 'backfill',\n",
" 'between_time',\n",
" 'bfill',\n",
" 'bool',\n",
" 'boxplot',\n",
" 'clip',\n",
" 'columns',\n",
" 'combine',\n",
" 'combine_first',\n",
" 'compare',\n",
" 'convert_dtypes',\n",
" 'copy',\n",
" 'corr',\n",
" 'corrwith',\n",
" 'count',\n",
" 'cov',\n",
" 'cummax',\n",
" 'cummin',\n",
" 'cumprod',\n",
" 'cumsum',\n",
" 'describe',\n",
" 'diff',
```



```
" 'div',\n",
" 'divide',\n",
" 'dot',\n",
" 'drop',\n",
" 'drop_duplicates',\n",
" 'droplevel',\n",
" 'dropna',\n",
" 'dtypes',\n",
" 'duplicated',\n",
" 'e1_1',\n",
" 'e1_10',\n",
" 'e1_11',\n",
" 'e1_12',\n",
" 'e1_13',\n",
" 'e1_14',\n",
" 'e1_15',\n",
" 'e1_2',\n",
" 'e1_3',\n",
" 'e1_4',\n",
" 'e1_5',\n",
" 'e1_6',\n",
" 'e1_7',\n",
" 'e1_8',\n",
" 'e1_9',\n",
" 'e1_n',\n",
" 'empty',\n",
" 'eq',\n",
" 'equals',\n",
" 'eval',\n",
" 'ewm',\n",
" 'expanding',\n",
" 'explode',\n",
" 'ffill',\n",
" 'fillna',\n",
" 'filter',\n",
" 'first',\n",
" 'first_valid_index',\n",
" 'flags',\n",
" 'floordiv',\n",
" 'from_dict',\n",
" 'from_records',\n",
" 'ge',\n",
" 'get',\n",
" 'groupby',\n",
" 'gt',\n",
" 'head',\n",
" 'hist',\n",
" 'iat',\n",
" 'idxmax',\n",
" 'idxmin',\n",
" 'iloc',\n",
" 'index',\n",
" 'infer_objects',\n",
" 'info',\n",
```

```
" 'insert',\n",  
" 'interpolate',\n",  
" 'isetitem',\n",  
" 'isin',\n",  
" 'isna',\n",  
" 'isnull',\n",  
" 'items',\n",  
" 'iteritems',\n",  
" 'iterrows',\n",  
" 'itertuples',\n",  
" 'join',\n",  
" 'keys',\n",  
" 'kurt',\n",  
" 'kurtosis',\n",  
" 'l1_1',\n",  
" 'l1_n',\n",  
" 'l2_1',\n",  
" 'l2_2',\n",  
" 'l2_3',\n",  
" 'l2_4',\n",  
" 'l2_5',\n",  
" 'l2_6',\n",  
" 'l2_7',\n",  
" 'l3_1',\n",  
" 'l3_10',\n",  
" 'l3_11',\n",  
" 'l3_12',\n",  
" 'l3_13',\n",  
" 'l3_14',\n",  
" 'l3_2',\n",  
" 'l3_3',\n",  
" 'l3_4',\n",  
" 'l3_5',\n",  
" 'l3_6',\n",  
" 'l3_7',\n",  
" 'l3_8',\n",  
" 'l3_9',\n",  
" 'l3_n',\n",  
" 'l4_1',\n",  
" 'l4_10',\n",  
" 'l4_11',\n",  
" 'l4_2',\n",  
" 'l4_3',\n",  
" 'l4_4',\n",  
" 'l4_5',\n",  
" 'l4_6',\n",  
" 'l4_7',\n",  
" 'l4_8',\n",  
" 'l4_9',\n",  
" 'l4_n',\n",  
" 'last',\n",  
" 'last_valid_index',\n",  
" 'le',\n",  
" 'loc',\n",
```

```
" 'lookup',\n",  
" 'lt',\n",  
" 'mad',\n",  
" 'mask',\n",  
" 'max',\n",  
" 'mean',\n",  
" 'median',\n",  
" 'melt',\n",  
" 'memory_usage',\n",  
" 'merge',\n",  
" 'min',\n",  
" 'mod',\n",  
" 'mode',\n",  
" 'mul',\n",  
" 'multiply',\n",  
" 'ndim',\n",  
" 'ne',\n",  
" 'nlargest',\n",  
" 'notna',\n",  
" 'notnull',\n",  
" 'nsmallest',\n",  
" 'nunique',\n",  
" 'pad',\n",  
" 'pct_change',\n",  
" 'pipe',\n",  
" 'pivot',\n",  
" 'pivot_table',\n",  
" 'plot',\n",  
" 'pop',\n",  
" 'pow',\n",  
" 'prod',\n",  
" 'product',\n",  
" 'q1_1',\n",  
" 'q1_n',\n",  
" 'quantile',\n",  
" 'query',\n",  
" 'radd',\n",  
" 'rank',\n",  
" 'rdiv',\n",  
" 'reindex',\n",  
" 'reindex_like',\n",  
" 'rename',\n",  
" 'rename_axis',\n",  
" 'reorder_levels',\n",  
" 'replace',\n",  
" 'resample',\n",  
" 'reset_index',\n",  
" 'rfloordiv',\n",  
" 'rmod',\n",  
" 'rmul',\n",  
" 'rolling',\n",  
" 'round',\n",  
" 'rpow',\n",  
" 'rsub',\n",
```

```
" 'rtruediv',\n",
" 'sample',\n",
" 'select_dtypes',\n",
" 'sem',\n",
" 'set_axis',\n",
" 'set_flags',\n",
" 'set_index',\n",
" 'shape',\n",
" 'shift',\n",
" 'size',\n",
" 'skew',\n",
" 'slice_shift',\n",
" 'sort_index',\n",
" 'sort_values',\n",
" 'squeeze',\n",
" 'stack',\n",
" 'std',\n",
" 'style',\n",
" 'sub',\n",
" 'subtract',\n",
" 'sum',\n",
" 'swapaxes',\n",
" 'swaplevel',\n",
" 'tail',\n",
" 'take',\n",
" 'to_clipboard',\n",
" 'to_csv',\n",
" 'to_dict',\n",
" 'to_excel',\n",
" 'to_feather',\n",
" 'to_gbq',\n",
" 'to_hdf',\n",
" 'to_html',\n",
" 'to_json',\n",
" 'to_latex',\n",
" 'to_markdown',\n",
" 'to_numpy',\n",
" 'to_orc',\n",
" 'to_parquet',\n",
" 'to_period',\n",
" 'to_pickle',\n",
" 'to_records',\n",
" 'to_sql',\n",
" 'to_stata',\n",
" 'to_string',\n",
" 'to_timestamp',\n",
" 'to_xarray',\n",
" 'to_xml',\n",
" 'transform',\n",
" 'transpose',\n",
" 'truediv',\n",
" 'truncate',\n",
" 'tz_convert',\n",
" 'tz_localize',
```

```

        " 'unstack',\n",
        " 'update',\n",
        " 'value_counts',\n",
        " 'values',\n",
        " 'var',\n",
        " 'where',\n",
        " 'xs']"
    ]
},
"execution_count": 30,
"metadata": {},
"output_type": "execute_result"
}
],
"source": [
    "dir(Data)"
]
},
{
    "cell_type": "code",
    "execution_count": 31,
    "metadata": {},
    "outputs": [
        {
            "data": {
                "text/html": [
                    "<div>\n",
                    "<style scoped>\n",
                    "    .dataframe tbody tr th:only-of-type {\n",
                    "        vertical-align: middle;\n",
                    "    }\n",
                    "\n",
                    "    .dataframe tbody tr th {\n",
                    "        vertical-align: top;\n",
                    "    }\n",
                    "\n",
                    "    .dataframe thead th {\n",
                    "        text-align: right;\n",
                    "    }\n",
                    "</style>\n",
                    "<table border='1' class='dataframe'>\n",
                    "    <thead>\n",
                    "        <tr style='text-align: right;'>\n",
                    "            <th></th>\n",
                    "            <th>l1_n</th>\n",
                    "            <th>l1_l</th>\n",
                    "            <th>l2_n</th>\n",
                    "            <th>l2_l</th>\n",
                    "            <th>l2_2</th>\n",
                    "            <th>l2_3</th>\n",
                    "            <th>l2_4</th>\n",
                    "            <th>l2_5</th>\n",
                    "            <th>l2_6</th>\n",
                    "            <th>l2_7</th>\n",

```

```

"      <th>...</th>\n",
"      <th>e1_6</th>\n",
"      <th>e1_7</th>\n",
"      <th>e1_8</th>\n",
"      <th>e1_9</th>\n",
"      <th>e1_10</th>\n",
"      <th>e1_11</th>\n",
"      <th>e1_12</th>\n",
"      <th>e1_13</th>\n",
"      <th>e1_14</th>\n",
"      <th>e1_15</th>\n",
"    </tr>\n",
"  </thead>\n",
"  <tbody>\n",
"    <tr>\n",
"      <th>1</th>\n",
"      <td>1</td>\n",
"      <td>10</td>\n",
"      <td>7</td>\n",
"      <td>0</td>\n",
"      <td>10</td>\n",
"      <td>9.5</td>\n",
"      <td>0.0</td>\n",
"      <td>10</td>\n",
"      <td>10</td>\n",
"      <td>0.0</td>\n",
"      <td>...</td>\n",
"      <td>0</td>\n",
"      <td>9</td>\n",
"      <td>9</td>\n",
"      <td>9</td>\n",
"      <td>9</td>\n",
"      <td>9</td>\n",
"      <td>10</td>\n",
"      <td>5</td>\n",
"      <td>0</td>\n",
"      <td>0</td>\n",
"    </tr>\n",
"    <tr>\n",
"      <th>2</th>\n",
"      <td>1</td>\n",
"      <td>10</td>\n",
"      <td>7</td>\n",
"      <td>10</td>\n",
"      <td>10</td>\n",
"      <td>0.0</td>\n",
"      <td>10.0</td>\n",
"      <td>10</td>\n",
"      <td>10</td>\n",
"      <td>10.0</td>\n",
"      <td>...</td>\n",
"      <td>9</td>\n",
"      <td>7</td>\n",
"      <td>9</td>\n",

```

```

"      <td>10</td>\n",
"      <td>10</td>\n",
"      <td>10</td>\n",
"      <td>5</td>\n",
"      <td>10</td>\n",
"      <td>5</td>\n",
"      <td>0</td>\n",
"    </tr>\n",
"  <tr>\n",
"    <th>3</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",
"    <td>7</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10.0</td>\n",
"    <td>9.5</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>9.5</td>\n",
"    <td>...</td>\n",
"    <td>0</td>\n",
"    <td>7</td>\n",
"    <td>5</td>\n",
"    <td>9</td>\n",
"    <td>9</td>\n",
"    <td>9</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"  </tr>\n",
"  <tr>\n",
"    <th>5</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",
"    <td>7</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>3.0</td>\n",
"    <td>7.0</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>9.0</td>\n",
"    <td>...</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>9</td>\n",
"    <td>8</td>\n",

```

```

"      <td>2</td>\n",
"    </tr>\n",
"  <tr>\n",
"    <th>6</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",
"    <td>7</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>3.0</td>\n",
"    <td>9.5</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>9.5</td>\n",
"    <td>...</td>\n",
"    <td>9</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>9</td>\n",
"    <td>5</td>\n",
"    <td>10</td>\n",
"    <td>8</td>\n",
"    <td>0</td>\n",
"  </tr>\n",
"  <tr>\n",
"    <th>7</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",
"    <td>7</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>0.0</td>\n",
"    <td>5.0</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>9.5</td>\n",
"    <td>...</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"  </tr>\n",
"  <tr>\n",
"    <th>9</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",

```



```

"      <td>7</td>\n",
"      <td>10</td>\n",
"      <td>10</td>\n",
"      <td>10.0</td>\n",
"      <td>9.5</td>\n",
"      <td>10</td>\n",
"      <td>10</td>\n",
"      <td>9.5</td>\n",
"      <td>...</td>\n",
"      <td>10</td>\n",
"      <td>7</td>\n",
"      <td>0</td>\n",
"      <td>9</td>\n",
"      <td>9</td>\n",
"      <td>9</td>\n",
"      <td>0</td>\n",
"      <td>5</td>\n",
"      <td>0</td>\n",
"      <td>0</td>\n",
"    </tr>\n",
"  <tr>\n",
"    <th>10</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",
"    <td>7</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>9.5</td>\n",
"    <td>0.0</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>0.0</td>\n",
"    <td>...</td>\n",
"    <td>9</td>\n",
"    <td>7</td>\n",
"    <td>9</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>0</td>\n",
"    <td>0</td>\n",
"  </tr>\n",
"  <tr>\n",
"    <th>12</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",
"    <td>7</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>9.5</td>\n",
"    <td>9.5</td>\n",
"    <td>10</td>\n",

```

```

"      <td>10</td>\n",
"      <td>9.5</td>\n",
"      <td>...</td>\n",
"      <td>9</td>\n",
"      <td>7</td>\n",
"      <td>9</td>\n",
"      <td>10</td>\n",
"      <td>10</td>\n",
"      <td>10</td>\n",
"      <td>10</td>\n",
"      <td>0</td>\n",
"      <td>0</td>\n",
"      <td>0</td>\n",
"    </tr>\n",
"  <tr>\n",
"    <th>13</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",
"    <td>7</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>0.0</td>\n",
"    <td>0.0</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>7.0</td>\n",
"    <td>...</td>\n",
"    <td>10</td>\n",
"    <td>9</td>\n",
"    <td>9</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>5</td>\n",
"    <td>10</td>\n",
"  </tr>\n",
"  <tr>\n",
"    <th>14</th>\n",
"    <td>1</td>\n",
"    <td>10</td>\n",
"    <td>7</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>9.5</td>\n",
"    <td>0.0</td>\n",
"    <td>10</td>\n",
"    <td>10</td>\n",
"    <td>0.0</td>\n",
"    <td>...</td>\n",
"    <td>0</td>\n",
"    <td>7</td>\n",
"    <td>0</td>\n",

```

```

"      <td>3</td>\n",
"      <td>3</td>\n",
"      <td>3</td>\n",
"      <td>0</td>\n",
"      <td>3</td>\n",
"      <td>0</td>\n",
"      <td>0</td>\n",
"    </tr>\n",
"  </tbody>\n",
"</table>\n",
"<p>11 rows Ã— 55 columns</p>\n",
"</div>"
],
"text/plain": [
...   e1_6  11_n  11_1  12_n  12_1  12_2  12_3  12_4  12_5  12_6  12_7
...   "1      1    10     7     0    10    9.5    0.0    10    10    0.0
...   0      \n",
...   "2      1    10     7    10    10    0.0   10.0    10    10   10.0
...   9      \n",
...   "3      1    10     7    10    10   10.0    9.5    10    10    9.5
...   0      \n",
...   "5      1    10     7    10    10    3.0    7.0    10    10    9.0
...   10     \n",
...   "6      1    10     7    10    10    3.0    9.5    10    10    9.5
...   9      \n",
...   "7      1    10     7    10    10    0.0    5.0    10    10    9.5
...   0      \n",
...   "9      1    10     7    10    10   10.0    9.5    10    10    9.5
...   10     \n",
...   "10     1    10     7    10    10    9.5    0.0    10    10    0.0
...   9      \n",
...   "12     1    10     7    10    10    9.5    9.5    10    10    9.5
...   9      \n",
...   "13     1    10     7    10    10    0.0    0.0    10    10    7.0
...   10     \n",
...   "14     1    10     7    10    10    9.5    0.0    10    10    0.0
...   0      \n",
...   "\n",
...   "      e1_7  e1_8  e1_9  e1_10  e1_11  e1_12  e1_13  e1_14  e1_15
\n",
...   "1      9     9     9      9      9     10      5      0      0
\n",
...   "2      7     9    10     10     10      5     10      5      0
\n",
...   "3      7     5     9      9      9      0      0      0      0
\n",
...   "5     10    10    10     10     10     10      9      8      2
\n",
...   "6      0     0    10     10      9      5     10      8      0
\n",
...   "7      0     0     0      0      0      0      0      0      0
\n",
...   "\n",

```

```

      "9      7      0      9      9      9      0      5      0      0
\n",
      "10     7      9     10     10     10     10     10     0      0
\n",
      "12     7      9     10     10     10     10      0      0      0
\n",
      "13     9      9     10     10     10     10     10      5     10
\n",
      "14     7      0      3      3      3      0      3      0      0
\n",
      "\n",
      "[11 rows x 55 columns]"
    ]
  },
  "execution_count": 31,
  "metadata": {},
  "output_type": "execute_result"
}
],
"source": [
  "Data[Data[\"l4_2\"]==10]"
]
},
{
  "cell_type": "code",
  "execution_count": 32,
  "metadata": {},
  "outputs": [
    {
      "data": {
        "text/plain": [
          "Index(['l1_n', 'l1_1', 'l2_n', 'l2_1', 'l2_2', 'l2_3', 'l2_4',
'l2_5', 'l2_6',\n",
          "      'l2_7', 'l3_n', 'l3_1', 'l3_2', 'l3_3', 'l3_4', 'l3_5',
'l3_6', 'l3_7',\n",
          "      'l3_8', 'l3_9', 'l3_10', 'l3_11', 'l3_12', 'l3_13',
'l3_14', 'l4_n',\n",
          "      'l4_1', 'l4_2', 'l4_3', 'l4_4', 'l4_5', 'l4_6', 'l4_7',
'l4_8', 'l4_9',\n",
          "      'l4_10', 'l4_11', 'q1_n', 'q1_1', 'e1_n', 'e1_1', 'e1_2',
'e1_3',\n",
          "      'e1_4', 'e1_5', 'e1_6', 'e1_7', 'e1_8', 'e1_9', 'e1_10',
'e1_11',\n",
          "      'e1_12', 'e1_13', 'e1_14', 'e1_15'],\n",
          "      dtype='object')]"
        ]
      },
      "execution_count": 32,
      "metadata": {},
      "output_type": "execute_result"
    }
  ],
  "source": [
    "Data.columns"
  ]
}

```

```

    ]
  },
  {
    "cell_type": "markdown",
    "metadata": {},
    "source": [
      "## Indexing\n",
      "\n",
      "We implemented `__getitem__` in our data classes to enable easy
access of our data. Lets take a closer look:"
    ]
  },
  {
    "cell_type": "code",
    "execution_count": 35,
    "metadata": {},
    "outputs": [],
    "source": [
      "class my_list:\n",
      "    def __init__(self,a_list):\n",
      "        self._list=a_list\n",
      "        \n",
      "    def __getitem__(self,key):\n",
      "        print(key)\n",
      "        pass\n",
      "        #return self._list[key]\n",
      "        "
    ]
  },
  {
    "cell_type": "markdown",
    "metadata": {},
    "source": [
      "We get a lot of nice functionality... but not everything:"
    ]
  },
  {
    "cell_type": "code",
    "execution_count": 36,
    "metadata": {},
    "outputs": [
      {
        "name": "stdout",
        "output_type": "stream",
        "text": [
          "1\n",
          "(1, 2)\n",
          "(1, 2, 3)\n",
          "slice(1, 2, None)\n"
        ]
      }
    ],
    "source": [
      "obj = my_list([5,5,5])\n",

```

```

        "\n",
        "obj[1]\n",
        "obj[1,2]\n",
        "obj[1,2,3]\n",
        "obj[1:2]"
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "Note slicing results in a `slice` object not the slice of the data.
Let's look at `slice` closer:"
    ]
},
{
    "cell_type": "code",
    "execution_count": 37,
    "metadata": {},
    "outputs": [
        {
            "data": {
                "text/plain": [
                    "slice(1, 2, 3)"
                ]
            },
            "execution_count": 37,
            "metadata": {},
            "output_type": "execute_result"
        }
    ],
    "source": [
        "slice(1,2,3)"
    ]
},
{
    "cell_type": "code",
    "execution_count": 38,
    "metadata": {},
    "outputs": [
        {
            "data": {
                "text/plain": [
                    "[ '__class__',\n",
                    "  '__delattr__',\n",
                    "  '__dir__',\n",
                    "  '__doc__',\n",
                    "  '__eq__',\n",
                    "  '__format__',\n",
                    "  '__ge__',\n",
                    "  '__getattribute__',\n",
                    "  '__gt__',\n",
                    "  '__hash__',\n",
                    "  '__init__',\n",

```

```

        " '__init_subclass__','\n",
        " '__le__','\n",
        " '__lt__','\n",
        " '__ne__','\n",
        " '__new__','\n",
        " '__reduce__','\n",
        " '__reduce_ex__','\n",
        " '__repr__','\n",
        " '__setattr__','\n",
        " '__sizeof__','\n",
        " '__str__','\n",
        " '__subclasshook__','\n",
        " 'indices','\n",
        " 'start','\n",
        " 'step','\n",
        " 'stop']"
    ]
},
"execution_count": 38,
"metadata": {},
"output_type": "execute_result"
}
],
"source": [
    "dir(slice)"
]
},
{
    "cell_type": "code",
    "execution_count": 39,
    "metadata": {},
    "outputs": [
        {
            "data": {
                "text/plain": [
                    "1"
                ]
            },
            "execution_count": 39,
            "metadata": {},
            "output_type": "execute_result"
        }
    ],
    "source": [
        "slice(1,2,3).start"
    ]
},
{
    "cell_type": "code",
    "execution_count": 40,
    "metadata": {},
    "outputs": [
        {
            "name": "stdout",

```

```

        "output_type": "stream",
        "text": [
            "slice(None, 1, None)\n"
        ]
    },
    ],
    "source": [
        "obj[:1]"
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "We can detect in `__getitem__` when we get a slice object and use it
accordingly:"
    ]
},
{
    "cell_type": "code",
    "execution_count": 41,
    "metadata": {},
    "outputs": [],
    "source": [
        "class my_list:\n",
        "    def __init__(self,a_list):\n",
        "        self._list=a_list\n",
        "\n",
        "    def __getitem__(self, key):\n",
        "        if isinstance(key, slice):\n",
        "            start = key.start or 0\n",
        "            stop = key.stop or len(self._list)\n",
        "            step = key.step or 1\n",
        "            return [self._list[i] for i in range(start, stop, step)]\n",
        "        elif isinstance(key, int):\n",
        "            return self._list[key]\n",
        "        elif isinstance(key, tuple):\n",
        "            raise NotImplementedError\n",
        "        else:\n",
        "            raise TypeError "
    ]
},
{
    "cell_type": "code",
    "execution_count": 42,
    "metadata": {},
    "outputs": [
        {
            "data": {
                "text/plain": [
                    "[3, 4, 5, 6]"
                ]
            },
            "execution_count": 42,

```



```

        "metadata": {},
        "output_type": "execute_result"
    }
],
"source": [
    "my_list([1,2,3,4,5,6,7])[2:6:1]"
]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "What if we want to do something more complicated, like handle both
`M[i][j]` and `M[i,j]` in the same way?"
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "## Lazy Evaluation\n",
        "\n",
        "Matrix multiplication can be a time consuming operation. What if we
only need a few elements of the result of a matrix multiplication? Can we
some how only compute the elements we need? This is where the concept of
lazy evaluation can come in handy. We have already seen that python
generators are a built-in mechanism for lazy evaluation. Here, we create
our own implementation.\n",
        "\n",
        "Recall the matrix multiplication formula:\n",
        "\n",
        "
$$C=A \cdot B: C_{ij} = \sum_k A_{ik} B_{kj}$$
\n",
        "\n",
        "Note that we actually compute every element of the resultant matrix
independently, but in a typical implemenation of multiplication, we'll
loop over all elements of resultant matrix:"
    ]
},
{
    "cell_type": "code",
    "execution_count": 5,
    "metadata": {},
    "outputs": [],
    "source": [
        "def zero_matrix(m,n):\n",
        "    return [ [0 for _ in range(m)] for _ in range(n)]\n",
        "\n",
        "def is_matrix(M):\n",
        "    if isinstance(M,list):\n",
        "        row_length=len(M[0])\n",
        "        for row in M:\n",
        "            if not row_length==len(row):\n",
        "                return False\n",
        "        else:\n",

```

```

        False\n",
        return True\n",
        \n",
    "\n",
    "def matrix_shape(M):\n",
    "    if is_matrix(M):\n",
    "        m=len(M)\n",
    "        n=len(M[0])\n",
    "        return m,n\n",
    "    else:\n",
    "        0,0\n",
    "\n",
    "def matrix_multiply(M1,M2):\n",
    "    m1,n1=matrix_shape(M1)\n",
    "    m2,n2=matrix_shape(M2)\n",
    "    \n",
    "    if n1==m2:\n",
    "        \n",
    "        M3=zero_matrix(m1,n2)\n",
    "        \n",
    "        # Loop over ALL elements of the resultant matrix\n",
    "        for i in range(m1):\n",
    "            for j in range(n2):\n",
    "                # Compute the element\n",
    "                for k in range(n1):\n",
    "                    M3[i][j]+=M1[i][k]*M2[k][j]\n",
    "            return M3\n",
    "        \n",
    "        return False"
    ]
},
{
    "cell_type": "code",
    "execution_count": 44,
    "metadata": {},
    "outputs": [
        {
            "data": {
                "text/plain": [
                    "[[5, 8], [8, 13]]"
                ]
            },
            "execution_count": 44,
            "metadata": {},
            "output_type": "execute_result"
        }
    ],
    "source": [
        "M1 = [ [ 1, 2 ] , [ 2, 3 ] ]\n",
        "M2 = [ [ 1, 2 ] , [ 2, 3 ] ]\n",
        "\n",
        "matrix_multiply(M1,M2)\n",
        "
    ]

```

```

},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "Instead we can create a new matrix class that holds the results of a
product of two matrices... and only computes the elements it needs."
    ]
},
{
    "cell_type": "code",
    "execution_count": 45,
    "metadata": {},
    "outputs": [],
    "source": [
        "class lazy_multiplied_matrix:\n",
        "    def __init__(self,M1,M2):\n",
        "        m1,n1=matrix_shape(M1)\n",
        "        m2,n2=matrix_shape(M2)\n",
        "        \n",
        "        assert(n1==m2)\n",
        "        \n",
        "        self._n1=n1\n",
        "        \n",
        "        self._m=m1\n",
        "        self._n=n2\n",
        "        \n",
        "        self._M1=M1\n",
        "        self._M2=M2\n",
        "        \n",
        "        # By default the resultant matrix will be composed of Nones,\n",
        "        # indicating that no element is computed.\n",
        "        self._M3= [ [None for _ in range(self._m)] for _ in\n",
        "range(self._n)]\n",
        "        \n",
        "        def __getitem__(self,key):\n",
        "            if isinstance(key,tuple):\n",
        "                i,j=key\n",
        "            else:\n",
        "                return None\n",
        "        \n",
        "        if self._M3[i][j]:\n",
        "            return self._M3[i][j]\n",
        "        else:\n",
        "            self._M3[i][j]=0.\n",
        "        \n",
        "            for k in range(self._n1):\n",
        "                self._M3[i][j]+=self._M1[i][k]*self._M2[k][j]\n",
        "            \n",
        "            return self._M3[i][j]\n",
        "        \n",
        "        def __str__(self):\n",

```

```

        "        return str(self._M3)\n",
        "        \n",
        "        __repr__ = __str__\n",
        "        \n",
        "        "
    ]
},
{
    "cell_type": "code",
    "execution_count": 46,
    "metadata": {},
    "outputs": [],
    "source": [
        "M3=lazy_multiplied_matrix(M1,M2)"
    ]
},
{
    "cell_type": "code",
    "execution_count": 47,
    "metadata": {},
    "outputs": [
        {
            "data": {
                "text/plain": [
                    "[[None, None], [None, None]]"
                ]
            },
            "execution_count": 47,
            "metadata": {},
            "output_type": "execute_result"
        }
    ],
    "source": [
        "M3"
    ]
},
{
    "cell_type": "code",
    "execution_count": 48,
    "metadata": {
        "scrolled": true
    },
    "outputs": [
        {
            "data": {
                "text/plain": [
                    "13.0"
                ]
            },
            "execution_count": 48,
            "metadata": {},
            "output_type": "execute_result"
        }
    ],
}
],

```

```

"source": [
  "M3[1,1]"
]
},
{
  "cell_type": "code",
  "execution_count": 49,
  "metadata": {},
  "outputs": [
    {
      "data": {
        "text/plain": [
          "[[None, None], [None, 13.0]]"
        ]
      },
      "execution_count": 49,
      "metadata": {},
      "output_type": "execute_result"
    }
  ],
  "source": [
    "M3"
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "Based on everything presented today, can you imagine how you could implement sparse matrices?"
  ]
}
],
"metadata": {
  "kernelspec": {
    "display_name": "Python 3 (ipykernel)",
    "language": "python",
    "name": "python3"
  },
  "language_info": {
    "codemirror_mode": {
      "name": "ipython",
      "version": 3
    },
    "file_extension": ".py",
    "mimetype": "text/x-python",
    "name": "python",
    "nbconvert_exporter": "python",
    "pygments_lexer": "ipython3",
    "version": "3.13.12"
  }
},
"nbformat": 4,
"nbformat_minor": 4

```

